

ARTIFICIAL INTELLIGENCE

Lecture Set 05
Informed Search

Dr. Maria Siddiqua
Assistant Professor
Department of AI & DS
FAST-NUCES Karachi
maria.siddiqua@nu.edu.pk

Course Contents

1. Introduction to AI
2. Agents and Environments
3. Problem Solving by Searching
4. Uninformed Search
- **5. Informed Search**
6. Local Search
7. Constraint Satisfaction Problems
8. Adversarial Search
9. Logical Agents
10. First-order Logic
11. Uncertain Knowledge and Reasoning
12. Supervised, Unsupervised, and Reinforcement Learning
13. Recent Trends in AI and Applications of AI Algorithms

Table of Contents

- Informed Search
- Heuristic
- Heuristic Search
- Heuristic Function
- Best First Search
- Greedy Best First Search
- A^* Search
- Iterative Deepening A^*
- Bidirectional A^*
- Recursive Best First Search
- Informed Search Evaluation
- Real Life Examples of Informed Search

Informed Search

- Uninformed search strategies can find solutions to problems by systematically generating new states and testing them against the goal.
- Unfortunately, these strategies are incredibly inefficient in most cases.
- Informed search uses problem-specific knowledge—can find solutions more efficiently.

Heuristic

- Heuristics are "rules of thumb", educated guesses, intuitive judgments or simply common sense.
- In the context of search algorithms, a heuristic is an estimate or rule of thumb that provides additional information about the problem domain to guide the search towards a solution more efficiently.
- With an educated guess a person considers what they have observed in the past, and applies that history to a situation where a more definite answer has not yet been decided.
- It allows a person to reach a conclusion without exhaustive research thereby contributing to the reduction of search in a problem-solving activity.

Heuristic Search

- Uses domain-dependent (heuristic) information in order to search the space more efficiently.

Ways of using heuristic information:

- Deciding which node to expand next, instead of doing the expansion in a strictly breadth-first or depth-first order.
- In the course of expanding a node, deciding which successor or successors to generate, instead of blindly generating all possible successors at one time.
- Deciding that certain nodes should be discarded, or pruned, from the search space.

Heuristic Search

- A moment's reflection will show ourselves constantly using heuristics in the course of our everyday lives.
- If the sky is grey we conclude that it would be better to put on a coat before going out.
- We book our holidays in August because that is when the weather is best.

Heuristic Function

- A heuristic function, $h(n)$, provides an estimate of the cost of the path from a given node to the closest goal state.
- Must be zero if node represents a goal state.

Best First Search

- Depth first search is good because it allows a solution to be found without all competing branches to be expanded.
- Breadth first search is good because it does not get trapped on dead-end paths.
- Best First Search combines the advantages of the two.
- At each step of the best first search process, we select the most promising of the nodes we have generated so far.
- This is done by applying an appropriate heuristic function to each of them.

Best First Search

- An algorithm in which a node is selected for expansion based on an evaluation function $f(n)$.
 - *Traditionally the node with the lowest evaluation function is selected*
 - *Not an accurate name...expanding the best node first would be a straight march to the goal.*
 - *Choose the node that appears to be the best*

Best First Search

- There is a whole family of Best-First Search algorithms with different evaluation functions.
- Each has a heuristic function $h(n)$.
- $h(n)$ = estimated cost of the cheapest path from node n to a goal node.
- Example: In route planning the estimate of the cost of the cheapest path might be the straight line distance between two cities.

Best First Search

- **$g(n)$** = Cost from the initial state to the current state n .
- **$h(n)$** = Estimated cost of the cheapest path from node n to a goal node.
- **$f(n)$** = Evaluation function to select a node for expansion (usually the lowest cost node).

Greedy Best First Search

- Greedy Best-First search tries to expand the node that is closest to the goal assuming it will lead to a solution quickly

$$f(n) = h(n)$$

- $f(n)$ = Function that gives an evaluation of the state.
- $h(n)$ = The cost of getting from the current state to a goal state.

Greedy Best First Search

- The Greedy Best-First-Search algorithm works as uniform cost search, except that it has some estimate (called a heuristic) of how far from the goal any node is.
- Instead of selecting the node closest to the starting point, it selects the node closest to the goal.
- Greedy Best-First-Search is not guaranteed to find a shortest path.
- However, it runs much quicker than uniform cost search algorithm because it uses the heuristic function to guide its way towards the goal very quickly.

Greedy Best First Search

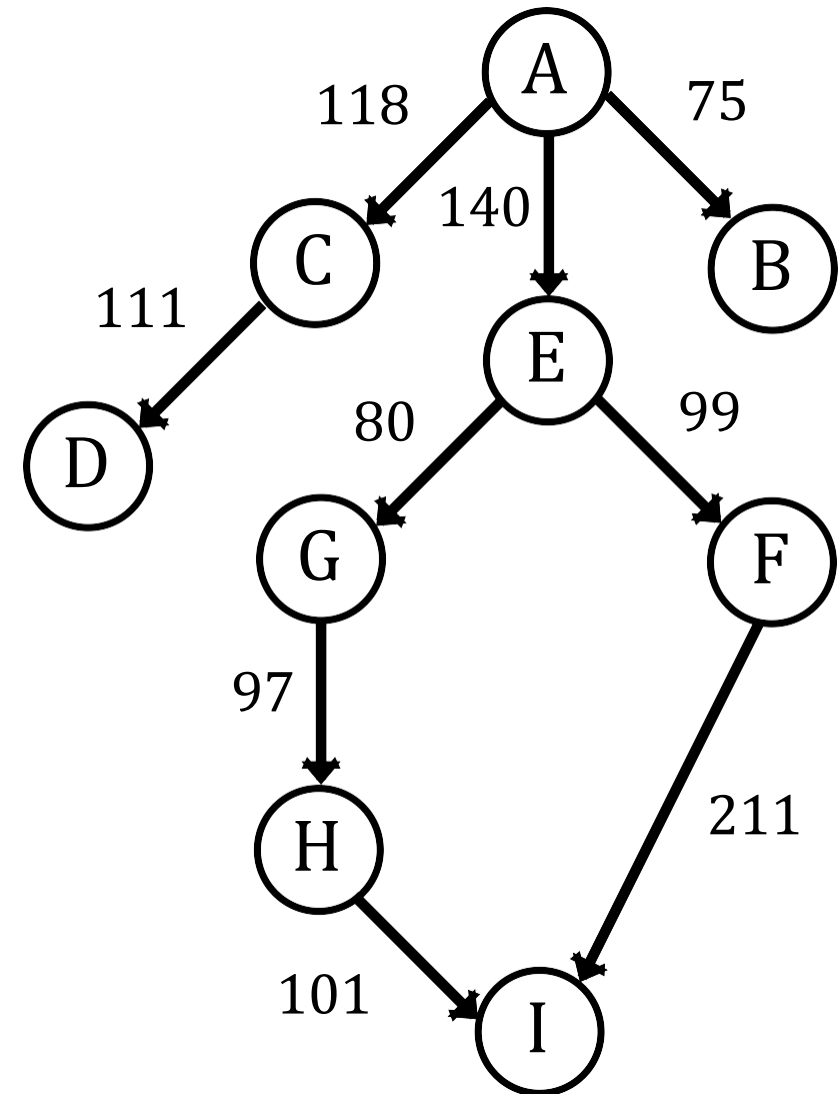
Algorithm:

- Step 01: Initialize Queue with Starting Node.
- Step 02: Pick the path with minimum heuristic cost $f(n)=h(n)$ from Queue.
- Step 03: If the minimum path is goal, stop algorithm you found the solution.
- Step 04: For each neighbor node v add expanded path $\langle v, P \rangle$ to Queue.
- Step 05: Repeat Step 02 to Step 04 until Queue is empty.

Greedy Best First Search (Problem 01)

■ A = Initial State

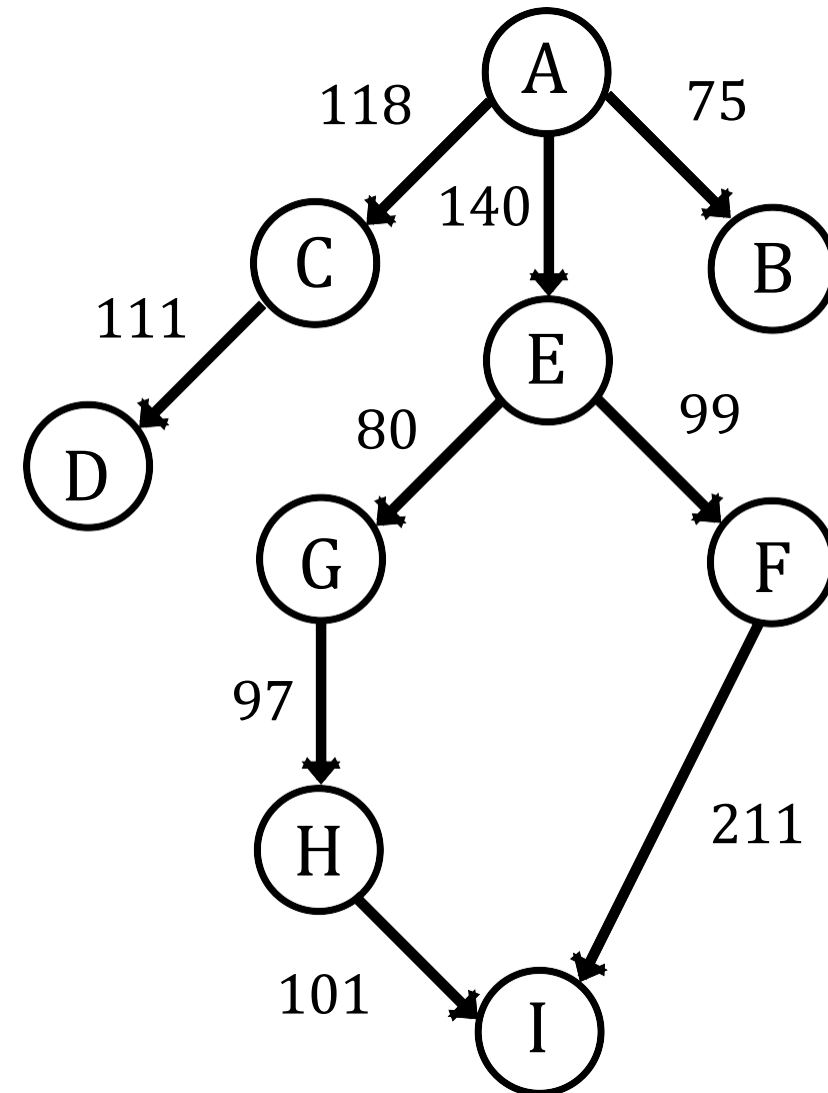
■ I = Goal State



Greedy Best First Search (Problem 01)

$f(n) = h(n) = \text{Straight Line Distance}$

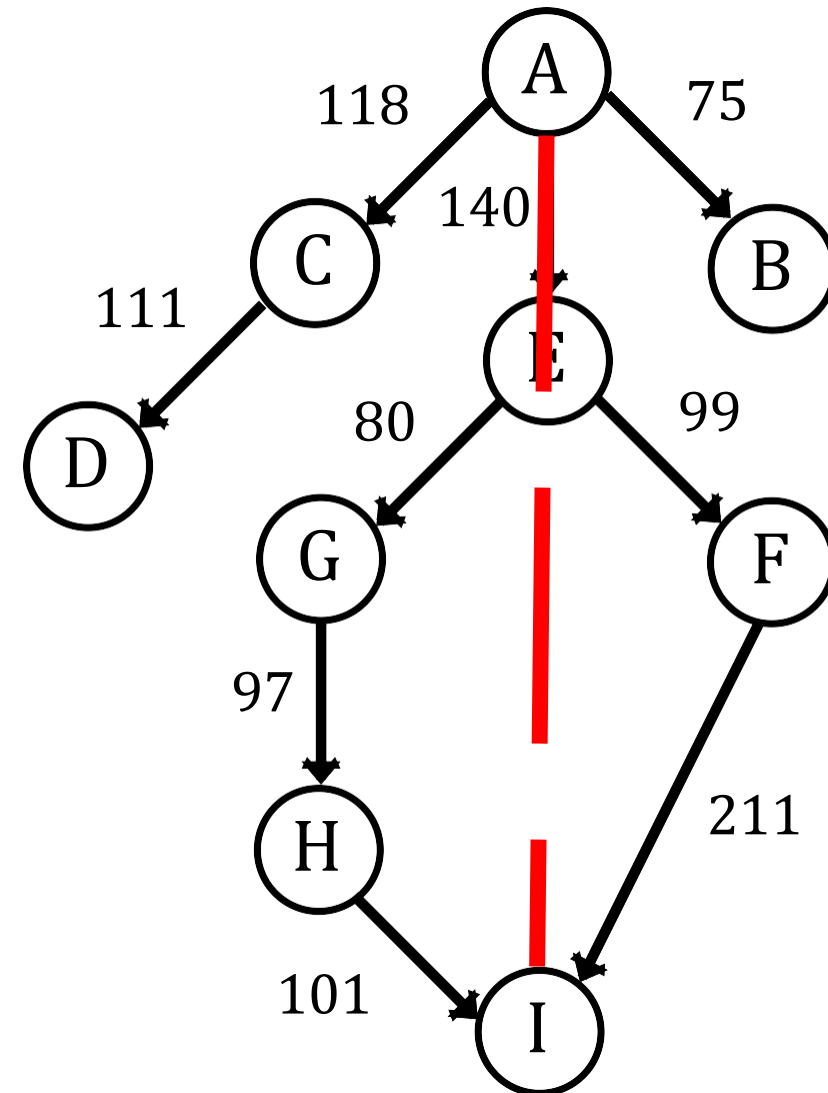
State	Heuristic: $h(n)$
A	366
B	374
C	329
D	244
E	253
F	178
G	193
H	98
I	0



Greedy Best First Search (Problem 01)

$f(n) = h(n) = \text{Straight Line Distance}$

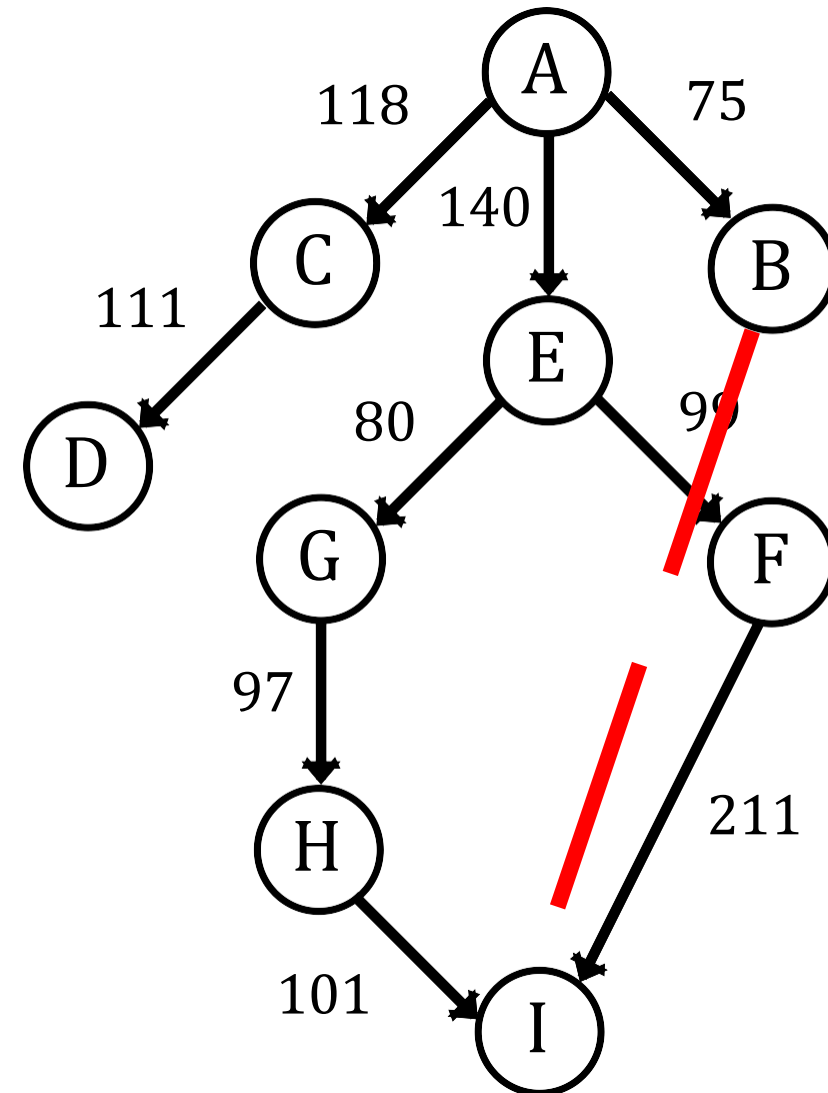
State	Heuristic: $h(n)$
A	366
B	374
C	329
D	244
E	253
F	178
G	193
H	98
I	0



Greedy Best First Search (Problem 01)

$f(n) = h(n) = \text{Straight Line Distance}$

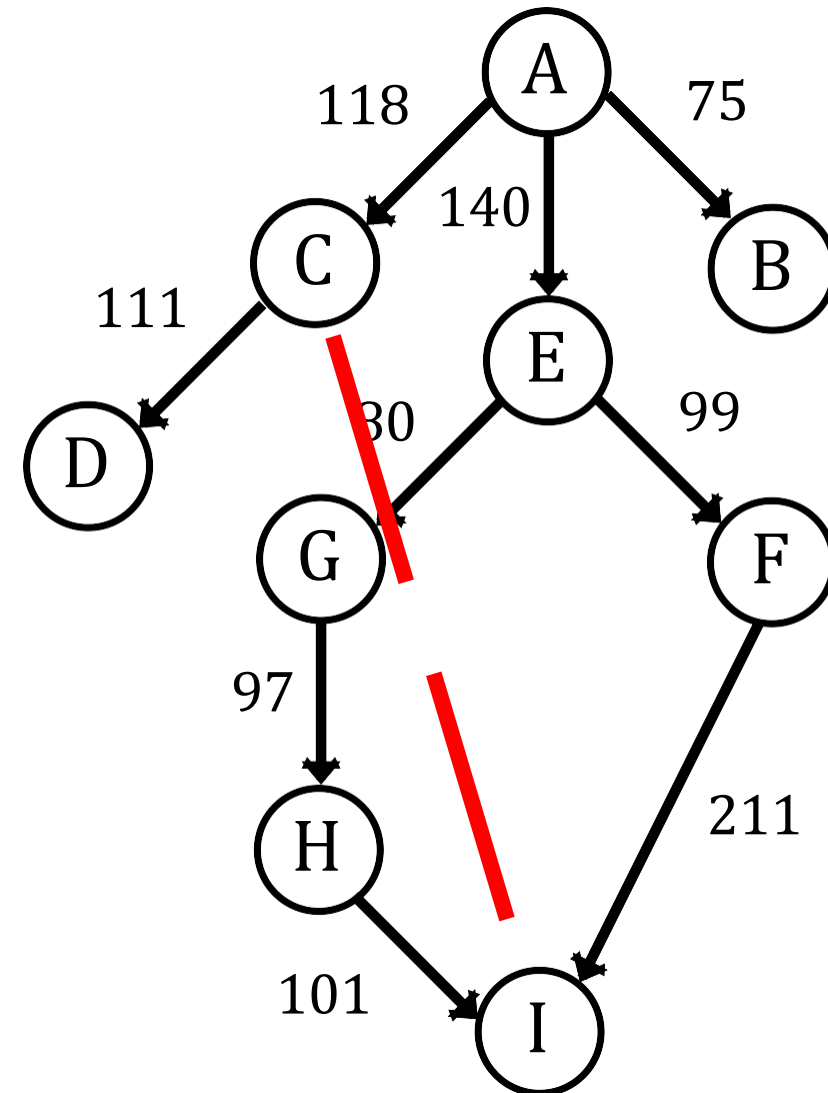
State	Heuristic: $h(n)$
A	366
B	374
C	329
D	244
E	253
F	178
G	193
H	98
I	0



Greedy Best First Search (Problem 01)

$f(n) = h(n) = \text{Straight Line Distance}$

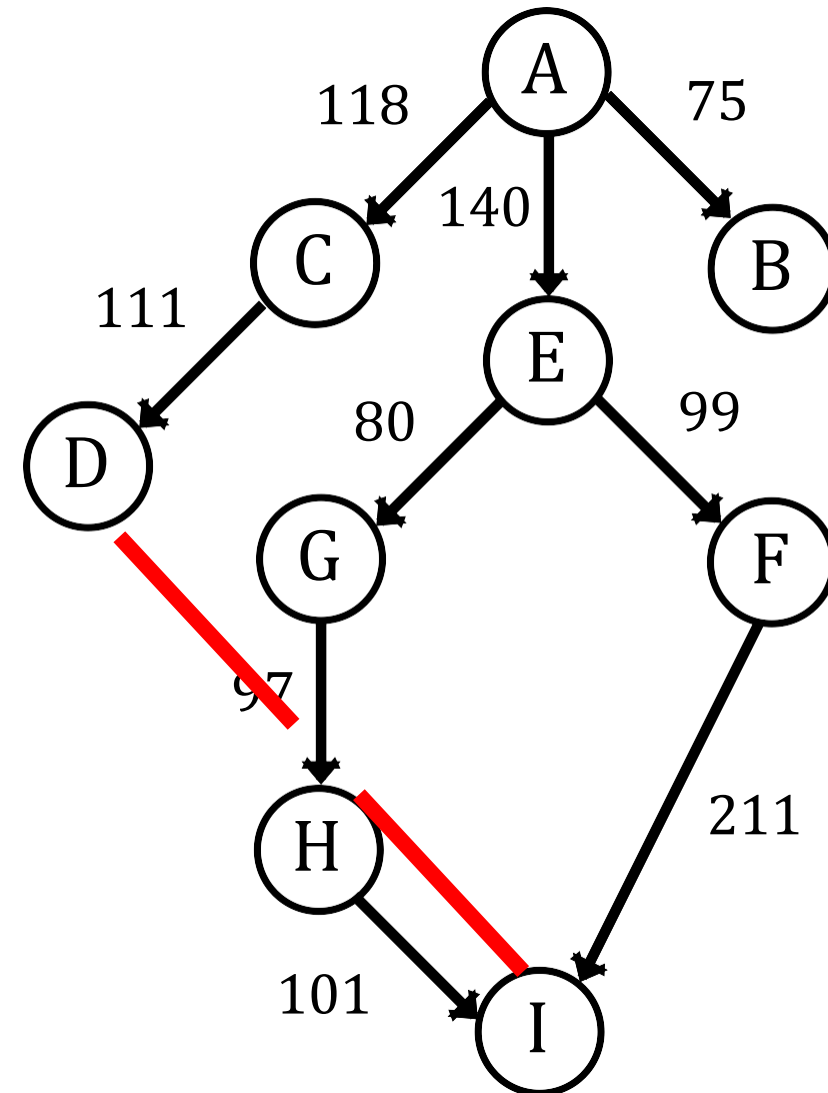
State	Heuristic: $h(n)$
A	366
B	374
C	329
D	244
E	253
F	178
G	193
H	98
I	0



Greedy Best First Search (Problem 01)

$f(n) = h(n) = \text{Straight Line Distance}$

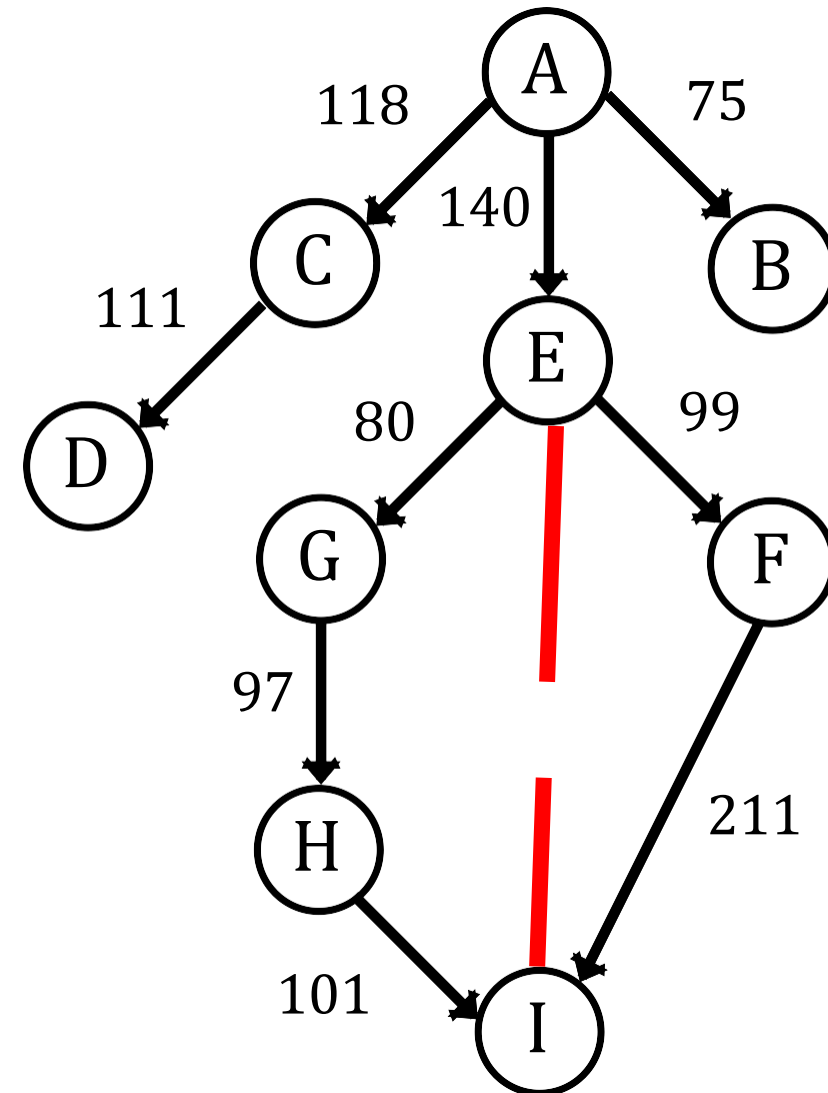
State	Heuristic: $h(n)$
A	366
B	374
C	329
D	244
E	253
F	178
G	193
H	98
I	0



Greedy Best First Search (Problem 01)

$f(n) = h(n) = \text{Straight Line Distance}$

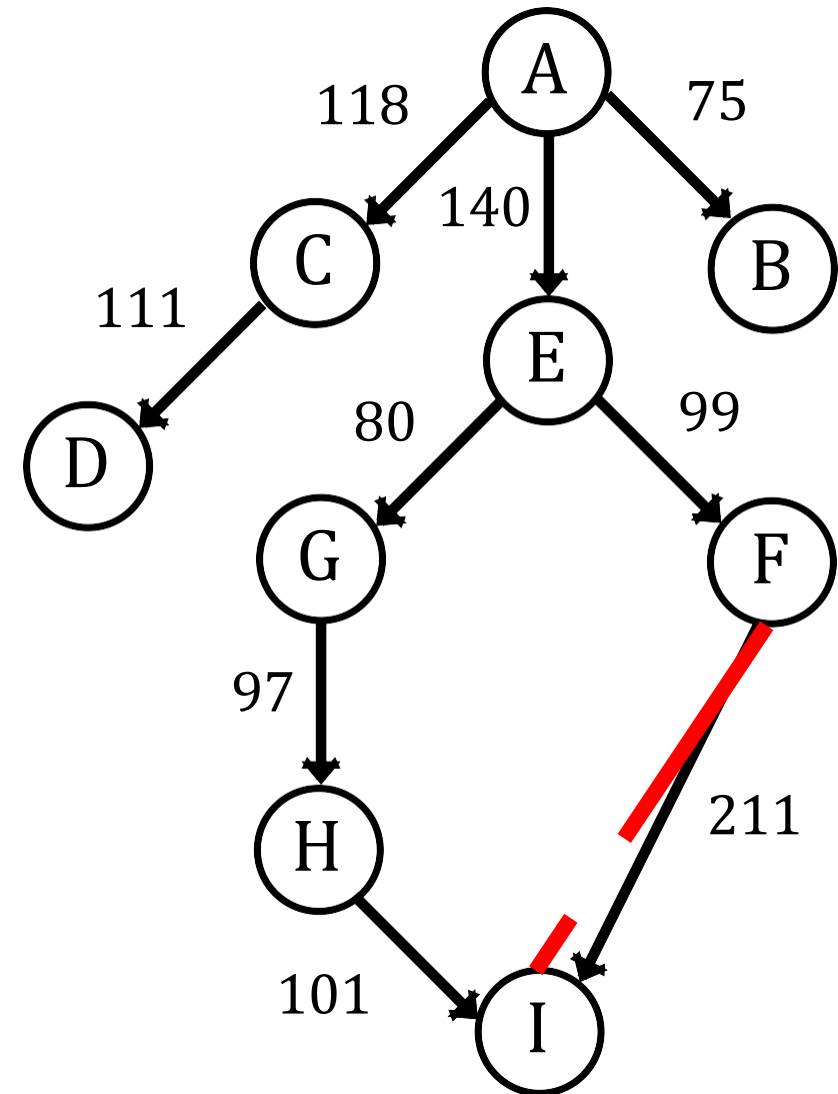
State	Heuristic: $h(n)$
A	366
B	374
C	329
D	244
E	253
F	178
G	193
H	98
I	0



Greedy Best First Search (Problem 01)

$f(n) = h(n) = \text{Straight Line Distance}$

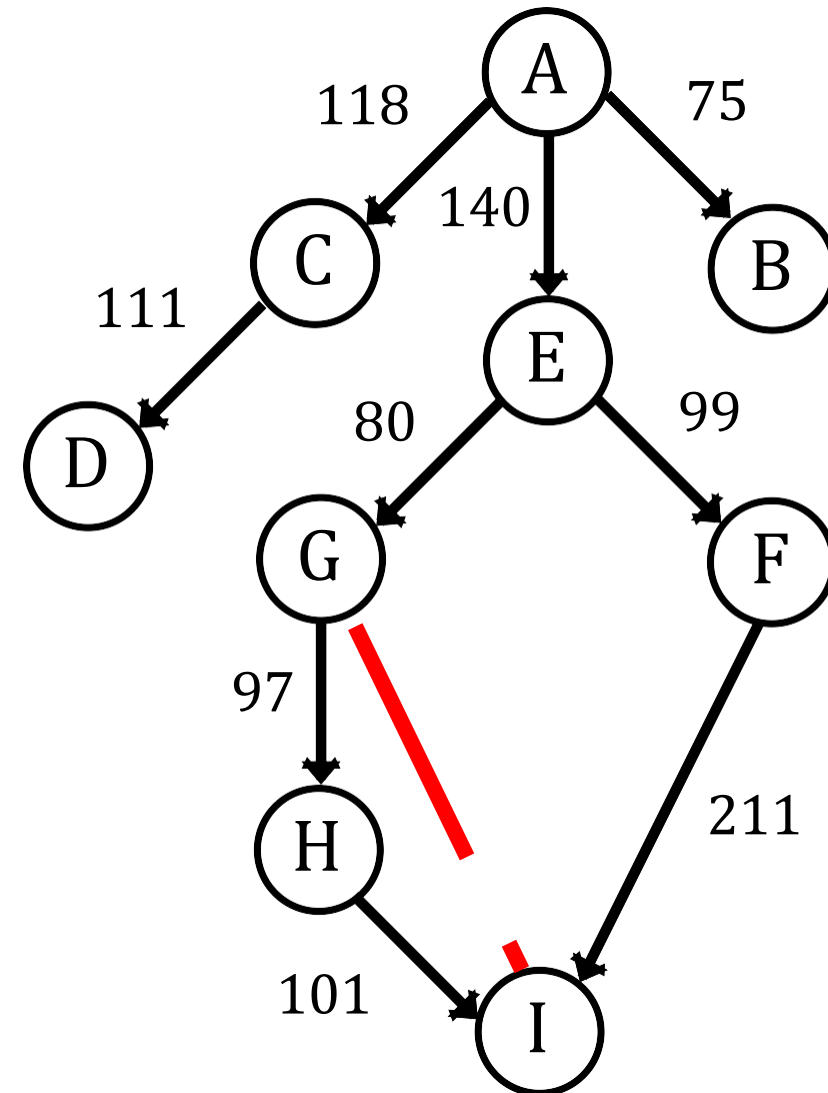
State	Heuristic: $h(n)$
A	366
B	374
C	329
D	244
E	253
F	178
G	193
H	98
I	0



Greedy Best First Search (Problem 01)

$f(n) = h(n) = \text{Straight Line Distance}$

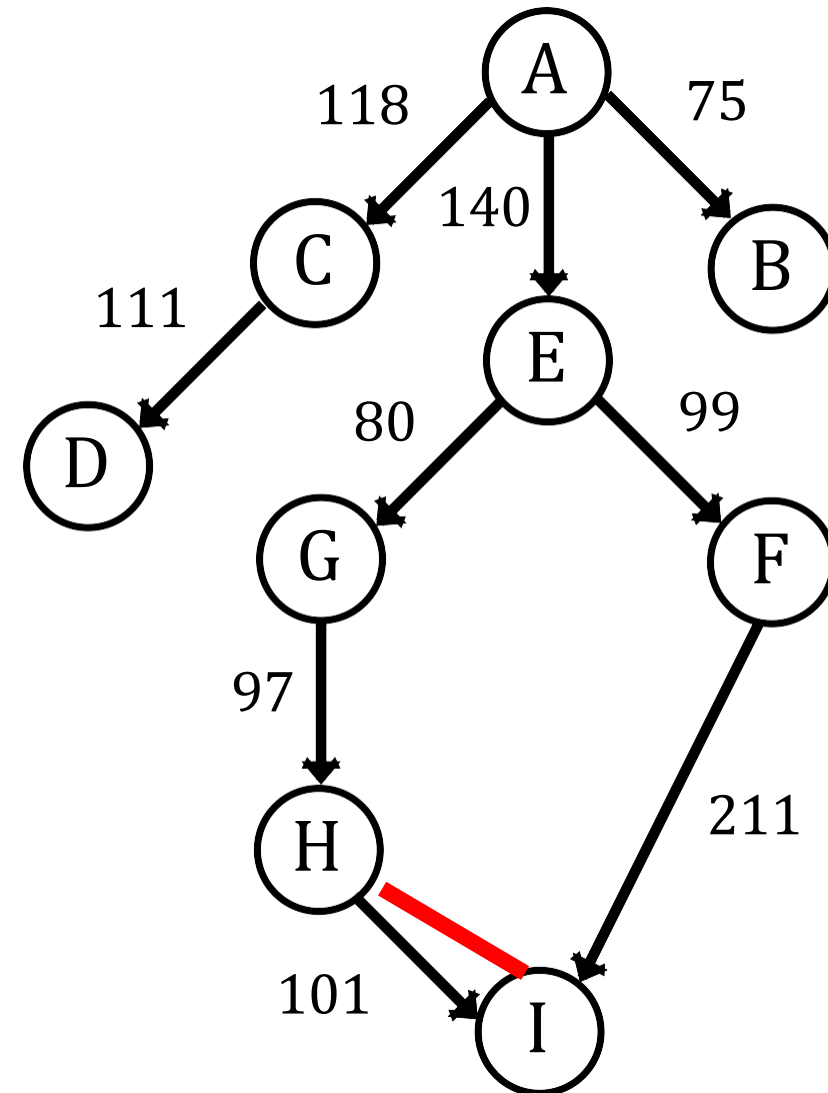
State	Heuristic: $h(n)$
A	366
B	374
C	329
D	244
E	253
F	178
G	193
H	98
I	0



Greedy Best First Search (Problem 01)

$f(n) = h(n) = \text{Straight Line Distance}$

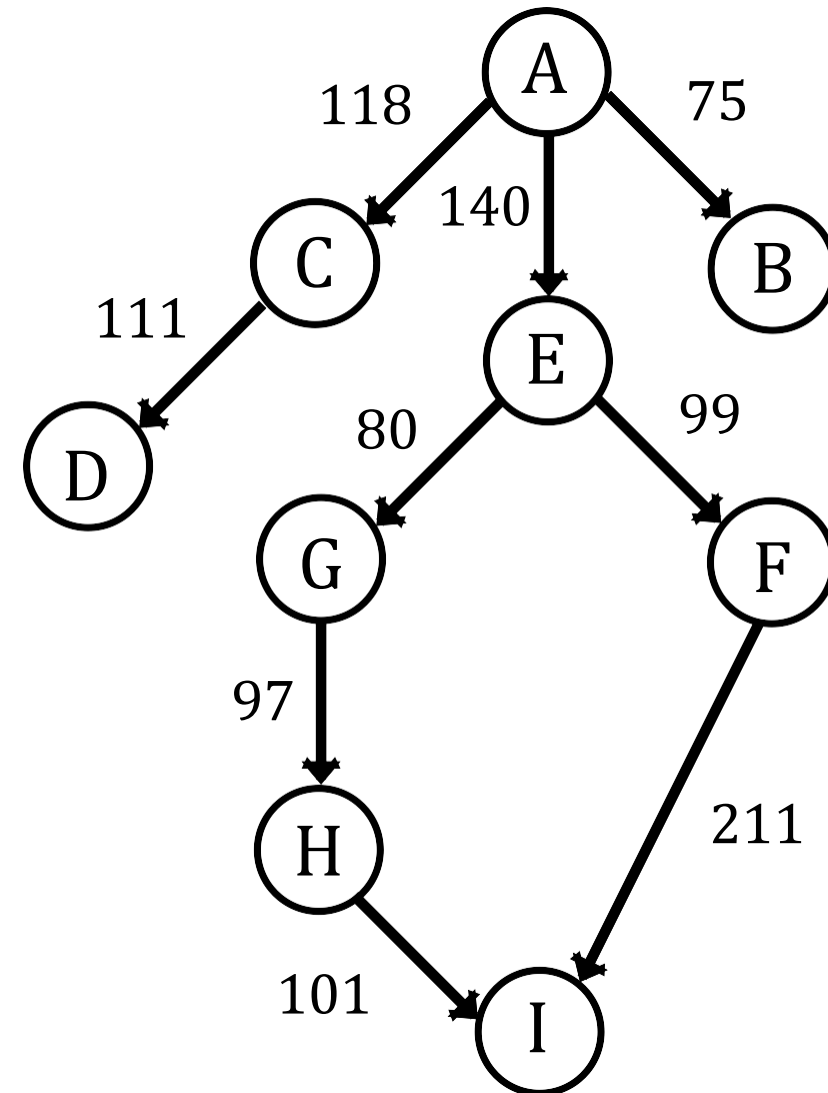
State	Heuristic: $h(n)$
A	366
B	374
C	329
D	244
E	253
F	178
G	193
H	98
I	0



Greedy Best First Search (Problem 01)

$f(n) = h(n) = \text{Straight Line Distance}$

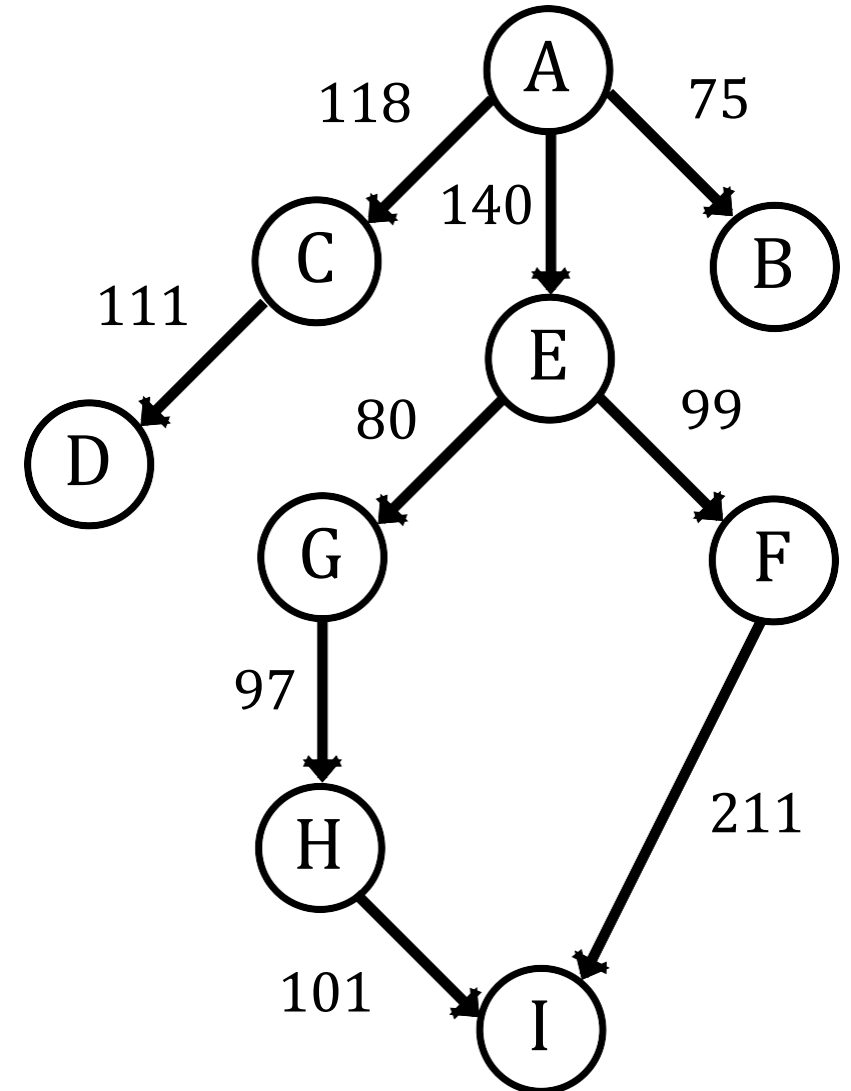
State	Heuristic: $h(n)$
A	366
B	374
C	329
D	244
E	253
F	178
G	193
H	98
I	0



Greedy Best First Search (Problem 01)

Queue		
Path	Cost	f(n)

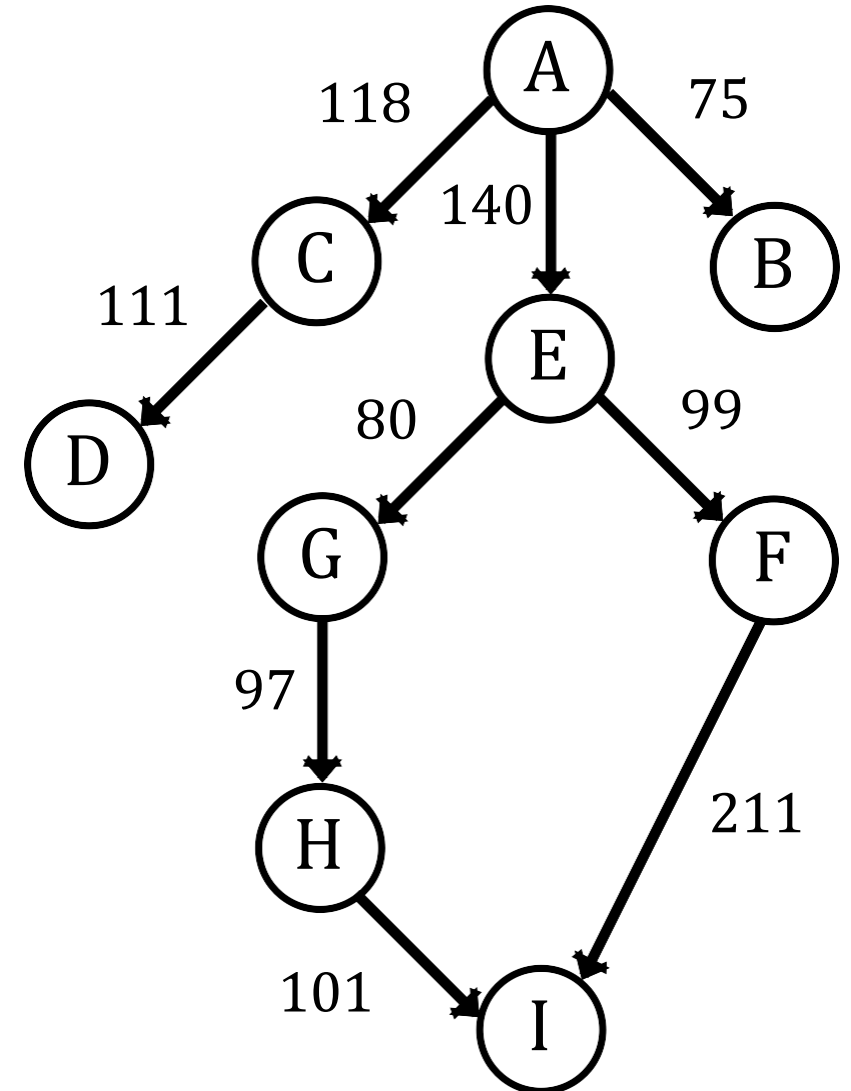
State	Heuristic: h(n)
A	366
B	374
C	329
D	244
E	253
F	178
G	193
H	98
I	0



Greedy Best First Search (Problem 01)

Queue		
Path	Cost	f(n)
<A>	0	366

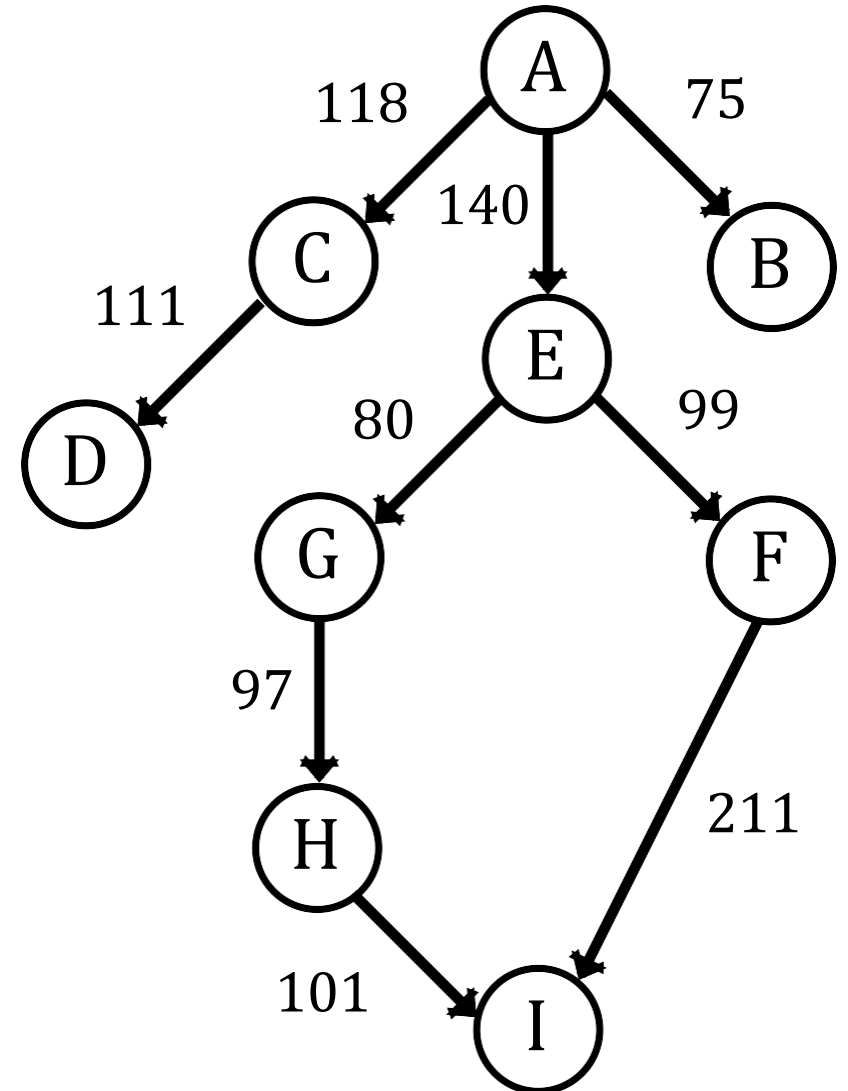
State	Heuristic: h(n)
A	366
B	374
C	329
D	244
E	253
F	178
G	193
H	98
I	0



Greedy Best First Search (Problem 01)

Queue		
Path	Cost	f(n)
<E,A>	140	253
<C,A>	118	329
<B,A>	75	374

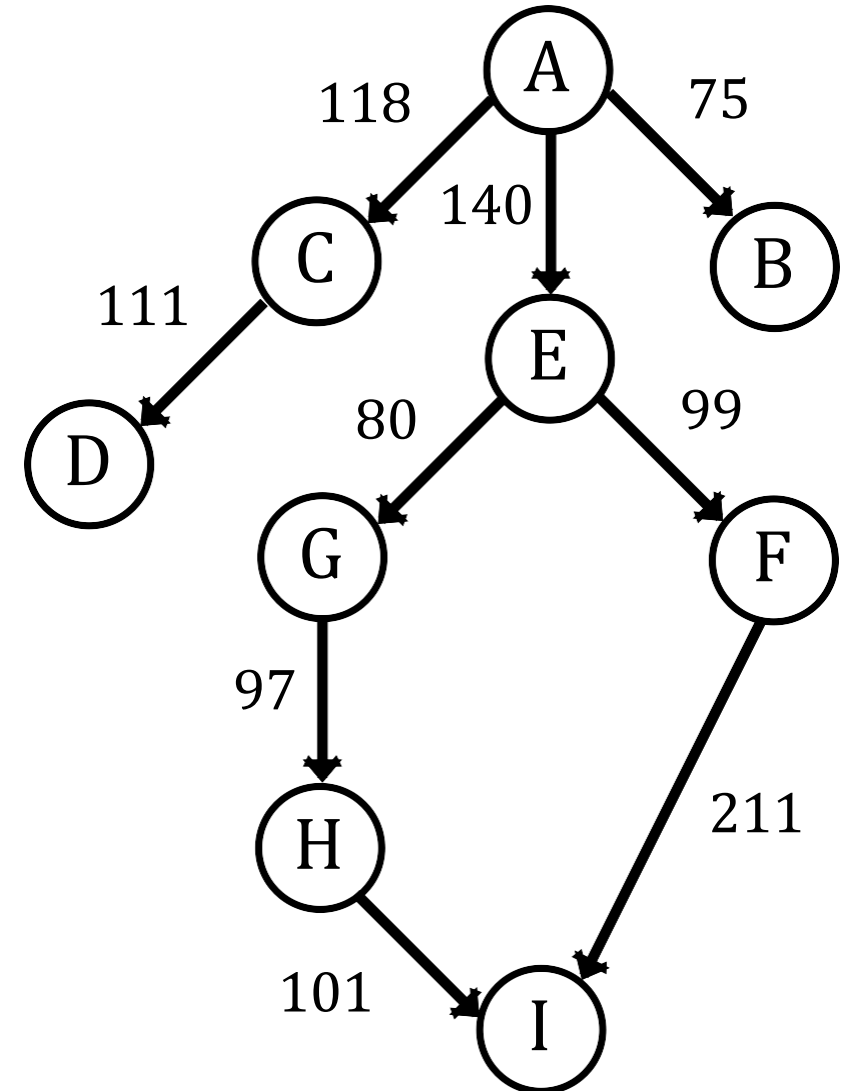
State	Heuristic: h(n)
A	366
B	374
C	329
D	244
E	253
F	178
G	193
H	98
I	0



Greedy Best First Search (Problem 01)

Queue		
Path	Cost	f(n)
<F,E,A>	239	178
<G,E,A>	220	193
<C,A>	118	329
<B,A>	75	374

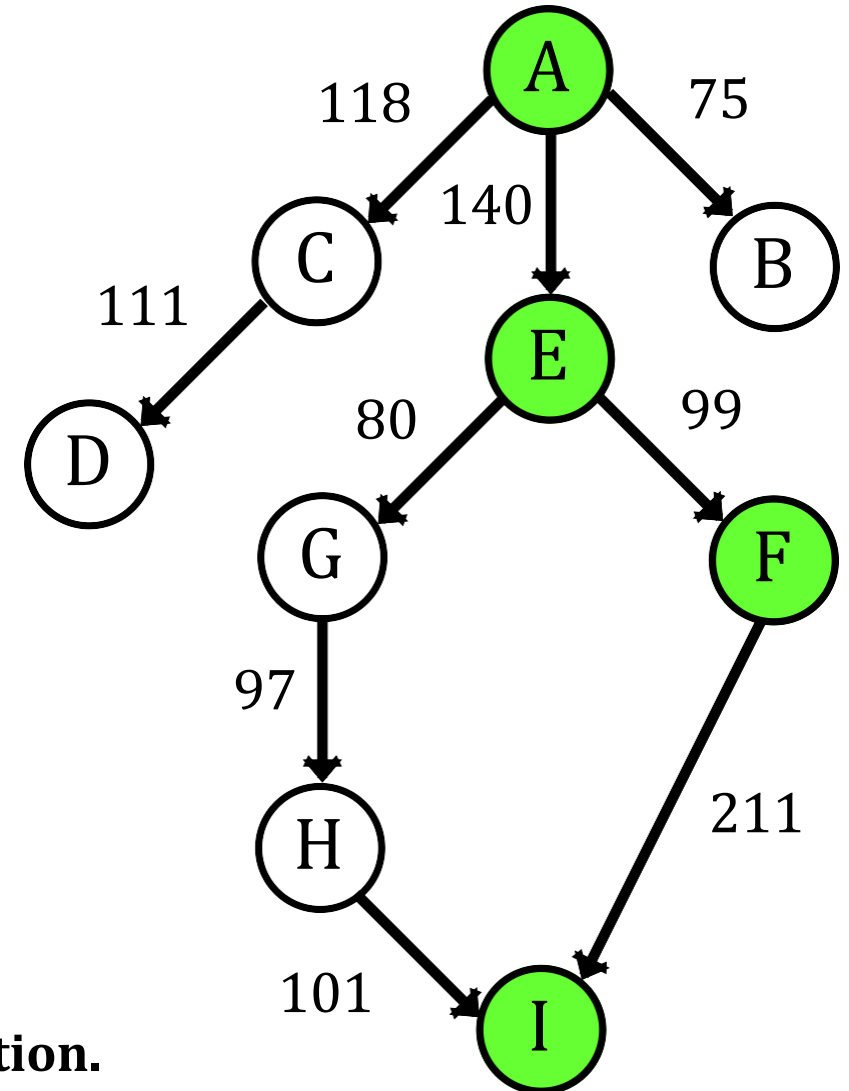
State	Heuristic: h(n)
A	366
B	374
C	329
D	244
E	253
F	178
G	193
H	98
I	0



Greedy Best First Search (Problem 01)

Queue		
Path	Cost	f(n)
<I,F,E,A>	450	0
<G,E,A>	220	193
<C,A>	118	329
<B,A>	75	374

State	Heuristic: h(n)
A	366
B	374
C	329
D	244
E	253
F	178
G	193
H	98
I	0



Solution = $A \rightarrow E \rightarrow F \rightarrow I$

Path Cost = $140 + 99 + 211 = 450$

As minimum path is the goal path, so we have found a solution.

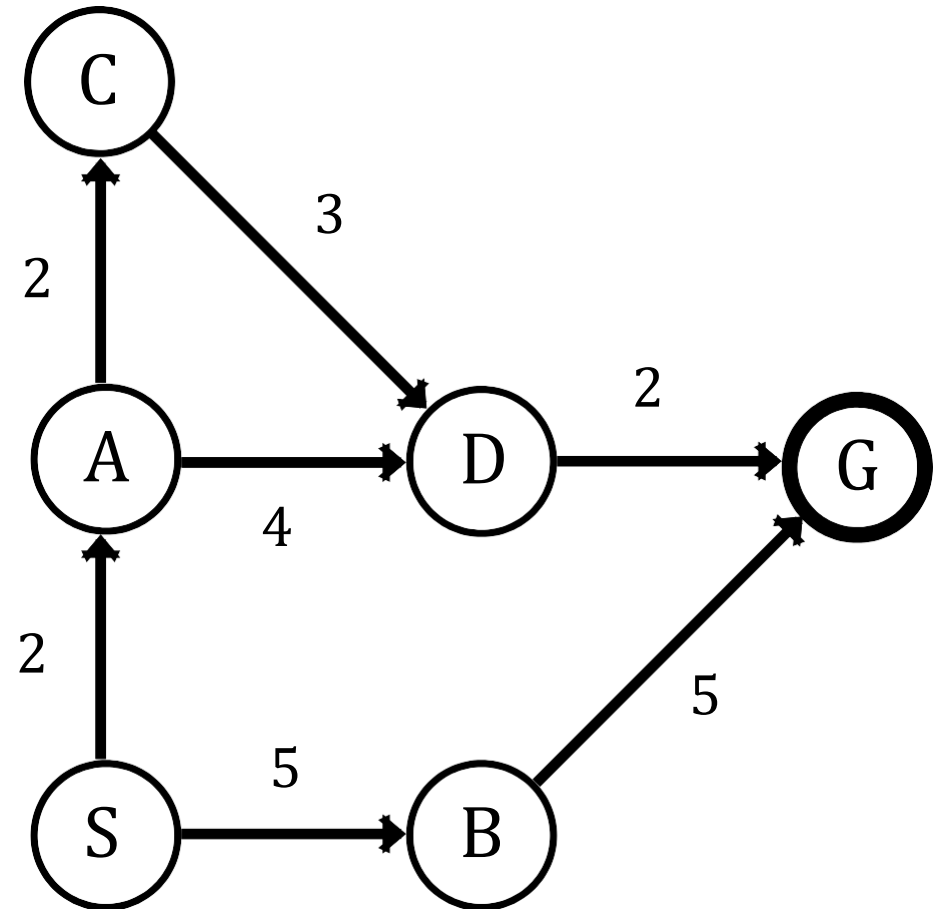
Greedy Best First Search

- Greedy First Search is not optimal.
- Path 1: $A \rightarrow E \rightarrow F \rightarrow I$ (Cost = $140+99+211=450$)
- Path 2: $A \rightarrow E \rightarrow G \rightarrow H \rightarrow I$ (Cost = $140+80+97+101=418$)
- There was path (i.e. path 2) more optimal than path 1.
- It is faster but not optimal.

Greedy Best First Search (Problem 02)

■ **S = Initial State**

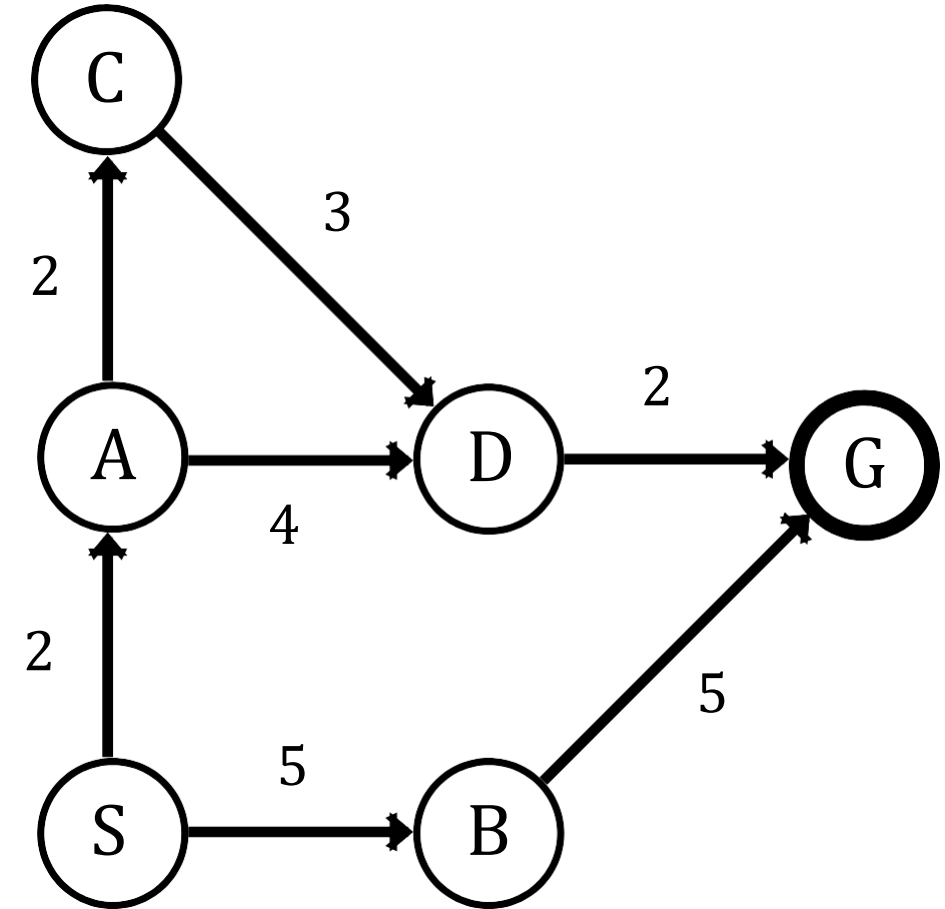
■ **G = Goal State**



Greedy Best First Search (Problem 02)

$f(n) = h(n) = \text{Straight Line Distance}$

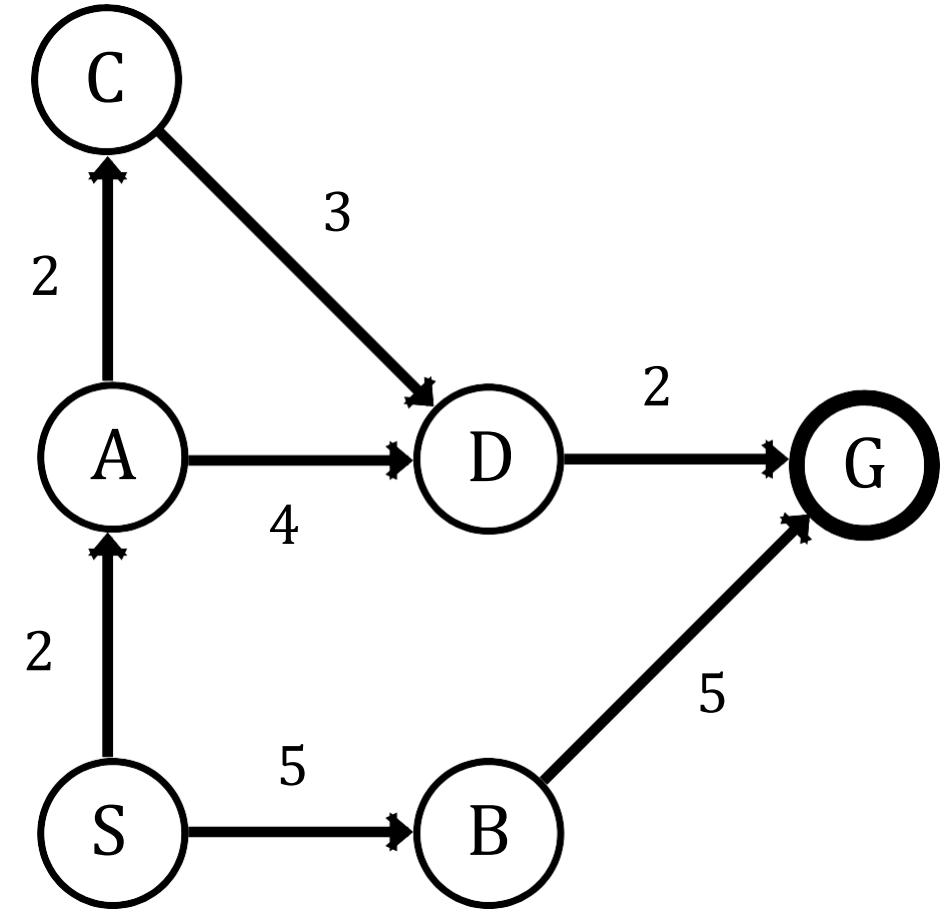
State	Heuristic: $h(n)$
S	10
A	2
B	3
C	1
D	4
G	0



Greedy Best First Search (Problem 02)

Queue		
Path	Cost	f(n)

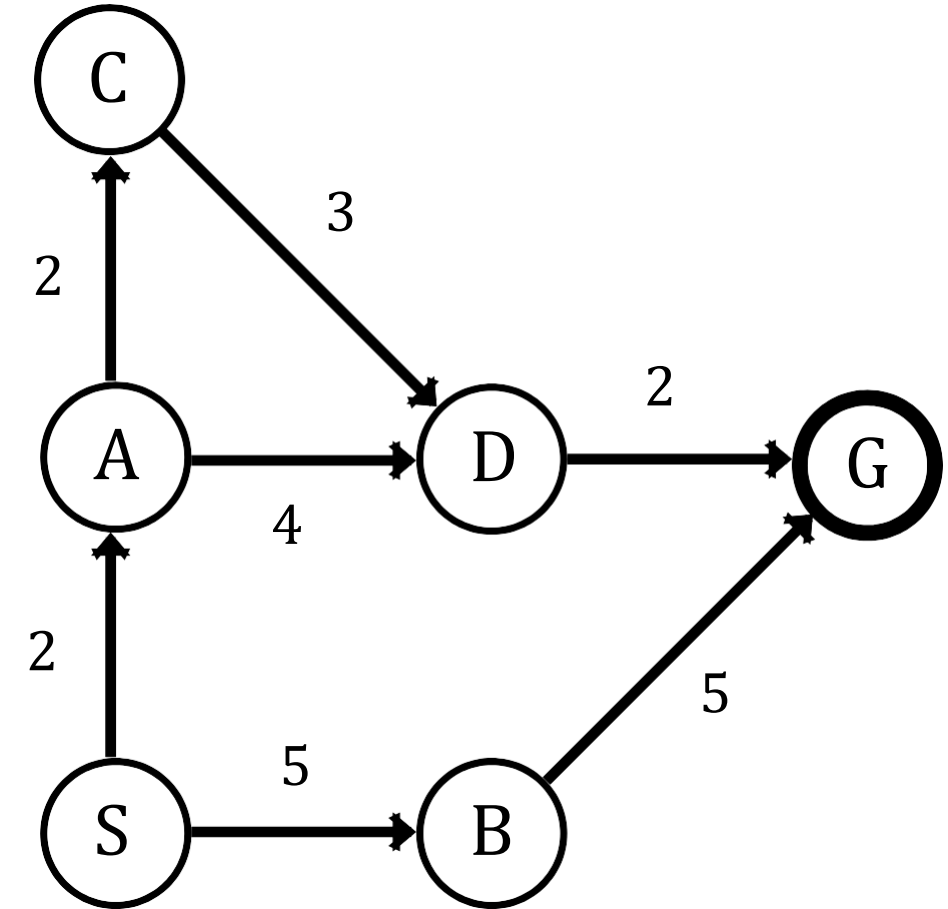
State	Heuristic: h(n)
S	10
A	2
B	3
C	1
D	4
G	0



Greedy Best First Search (Problem 02)

Queue		
Path	Cost	f(n)
<S>	0	10

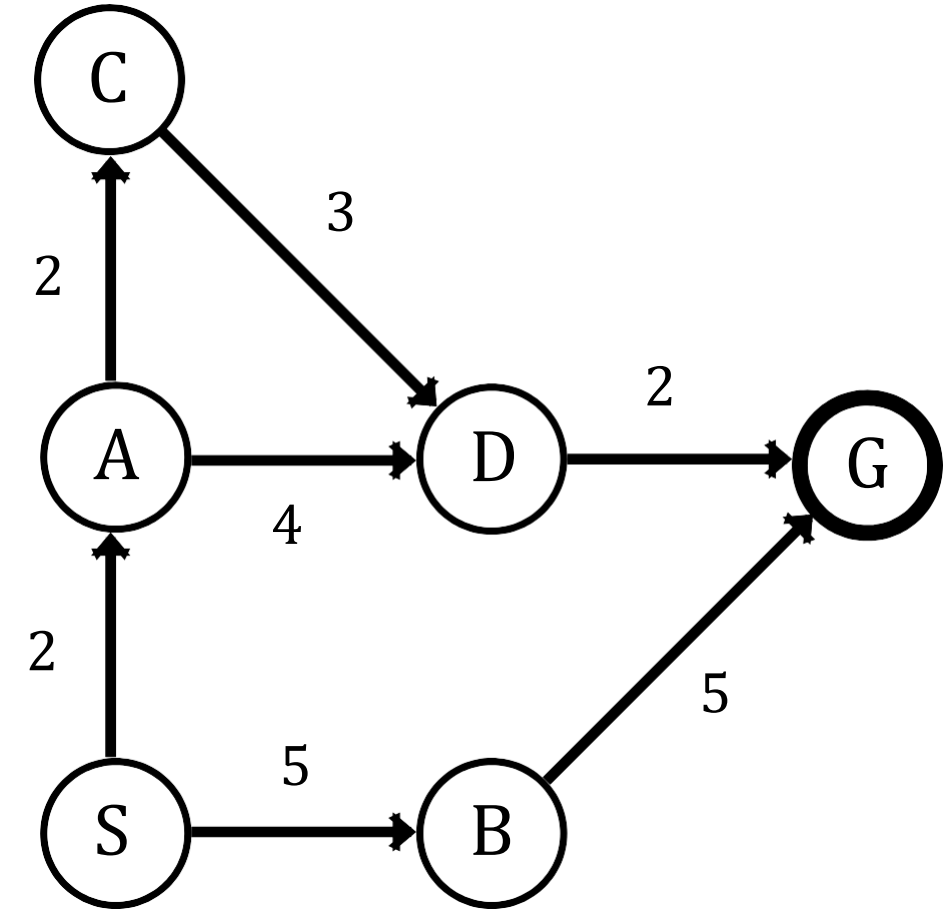
State	Heuristic: h(n)
S	10
A	2
B	3
C	1
D	4
G	0



Greedy Best First Search (Problem 02)

Queue		
Path	Cost	f(n)
<A,S>	2	2
<B,S>	5	3

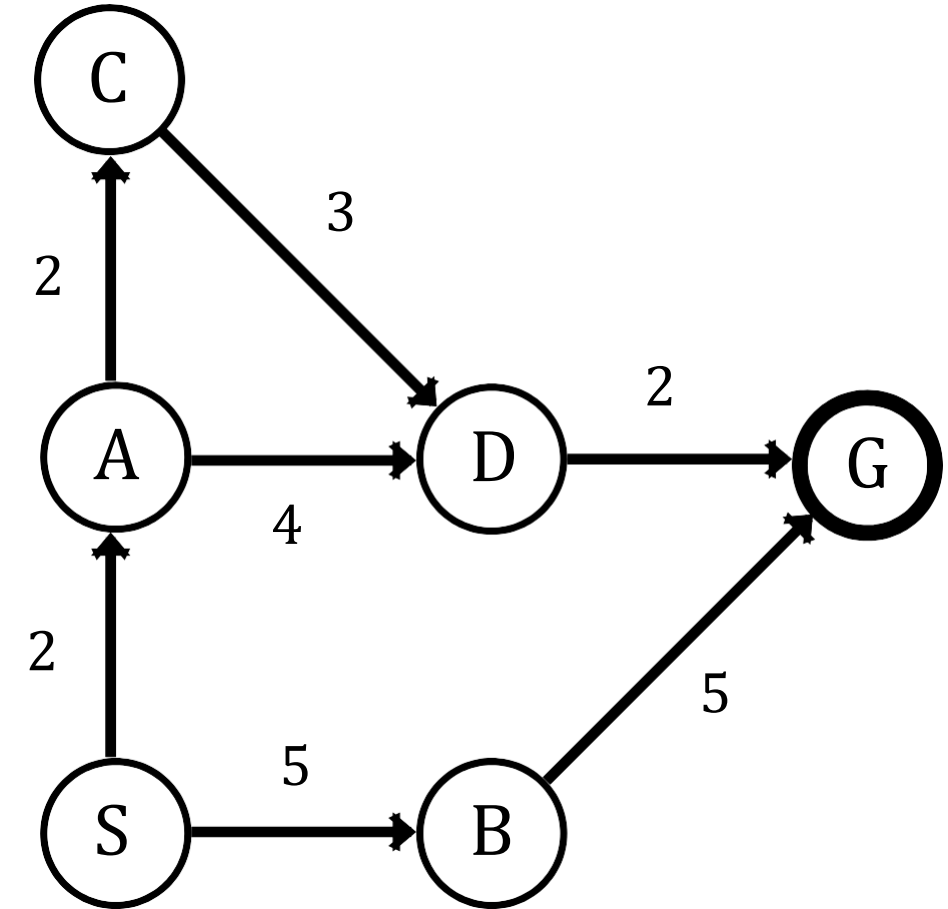
State	Heuristic: h(n)
S	10
A	2
B	3
C	1
D	4
G	0



Greedy Best First Search (Problem 02)

Queue		
Path	Cost	f(n)
<C,A,S>	4	1
<B,S>	5	3
<D,A,S>	6	4

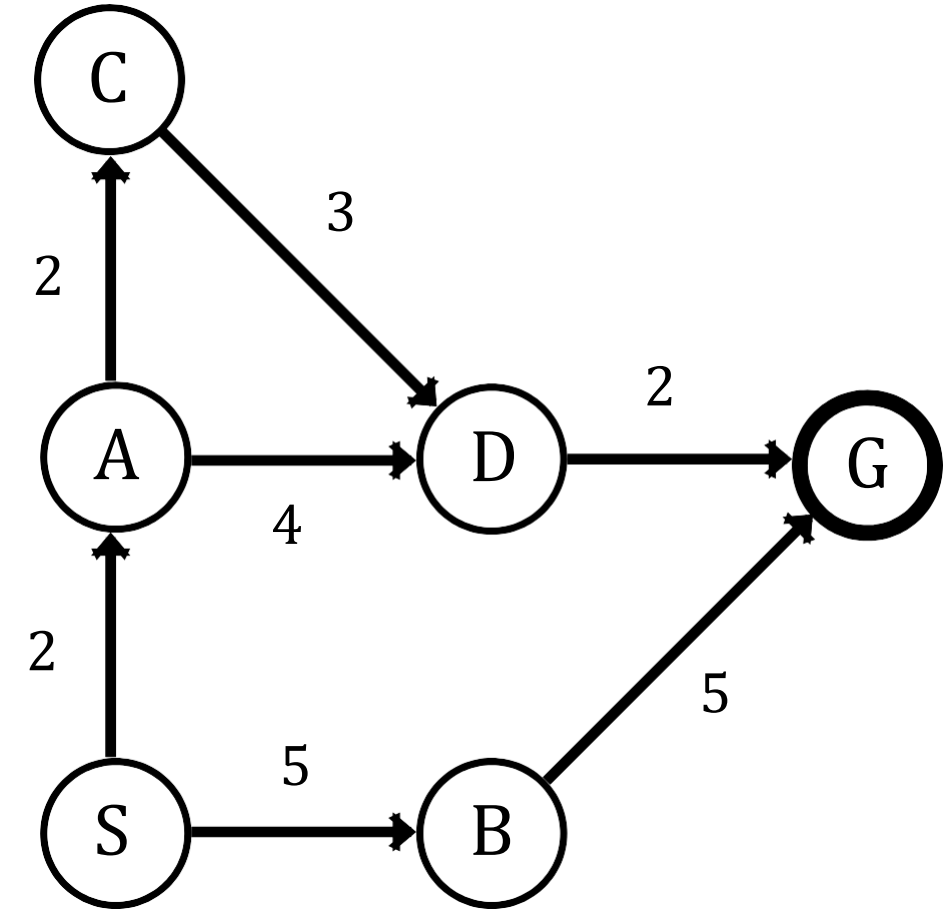
State	Heuristic: h(n)
S	10
A	2
B	3
C	1
D	4
G	0



Greedy Best First Search (Problem 02)

Queue		
Path	Cost	f(n)
<B,S>	5	3
<D,A,S>	6	4
<D,C,A,S>	7	4

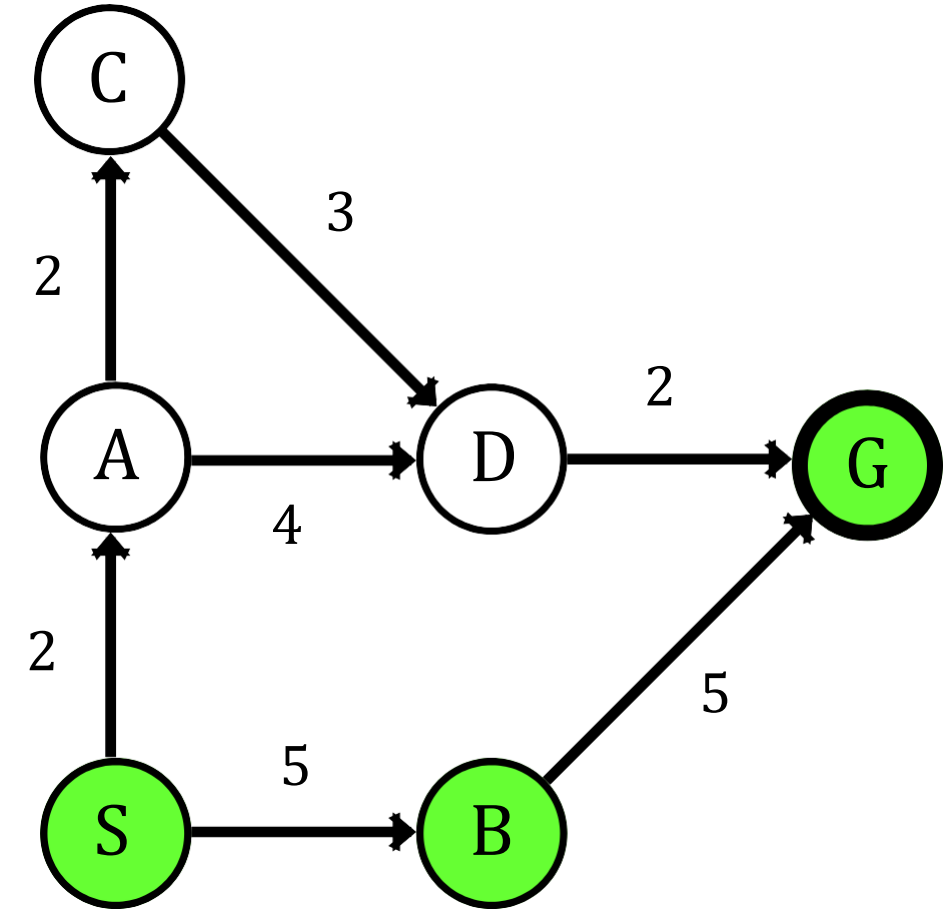
State	Heuristic: h(n)
S	10
A	2
B	3
C	1
D	4
G	0



Greedy Best First Search (Problem 02)

Queue		
Path	Cost	f(n)
<G,B,S>	10	0
<D,A,S>	6	4
<D,C,A,S>	7	4

State	Heuristic: h(n)
S	10
A	2
B	3
C	1
D	4
G	0



Solution = $S \rightarrow B \rightarrow G$
Path Cost = $5 + 5 = 10$

As minimum path is the goal path, so we have found a solution.

A* Search

- Greedy Search minimizes a heuristic $h(n)$ which is an estimated cost from a node n to the goal state. However, although greedy search can considerably cut the search time (efficient), it is neither optimal nor complete.
- Uniform Cost Search minimizes the cost $g(n)$ from the initial state to n . UCS is optimal and complete but not efficient.
- New Strategy: Combine Greedy Search and UCS to get an efficient algorithm which is complete and optimal.

A* Search

- A search algorithm to find the shortest path through a search space to a goal state using a heuristic.

$$f(n) = g(n) + h(n)$$

- $f(n)$ = Function that gives an evaluation of the state.
- $g(n)$ = The cost of getting from the initial state to the current state.
- $h(n)$ = The cost of getting from the current state to a goal state.

A* Search

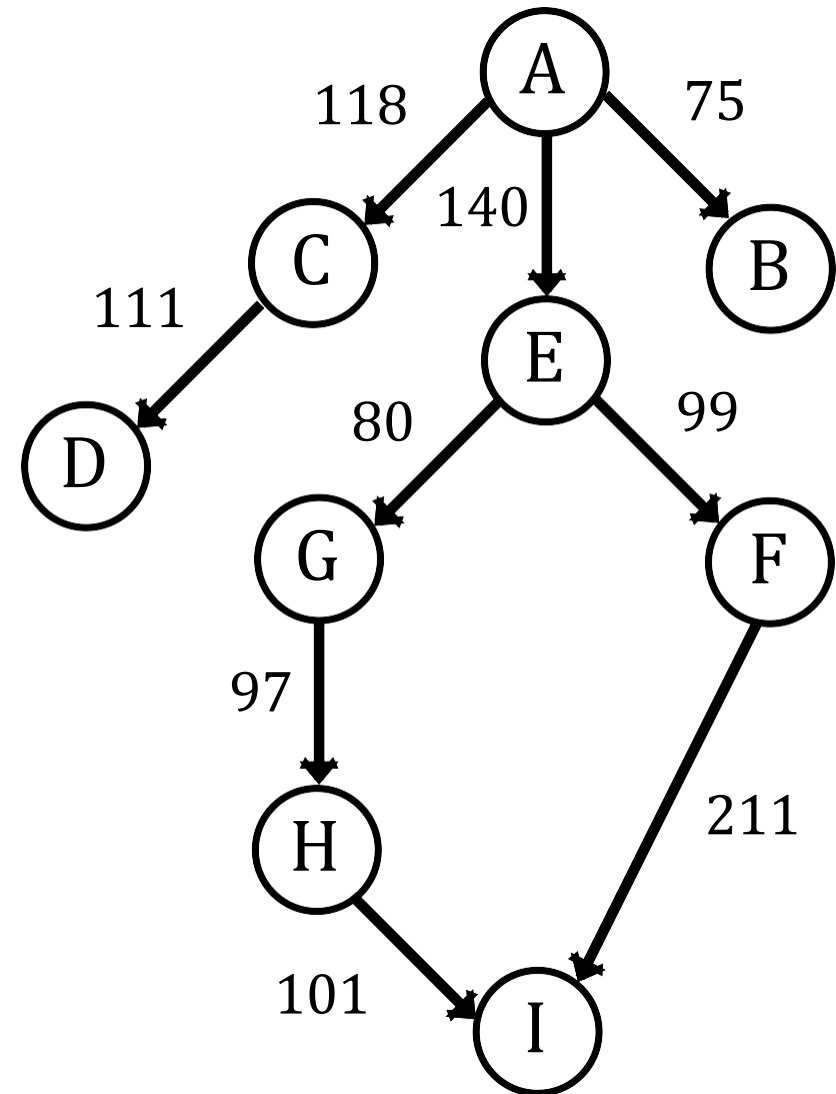
Algorithm:

- Step 01: Initialize Queue with Starting Node.
- Step 02: Pick the path with minimum heuristic cost $f(n)=g(n)+h(n)$ from Queue.
- Step 03: If the minimum path is goal, stop algorithm you found the solution.
- Step 04: For each neighbor node v add expanded path $\langle v, P \rangle$ to Queue.
- Step 05: Repeat Step 02 to Step 04 until Queue is empty.

A* Search (Problem 01)

■ A = Initial State

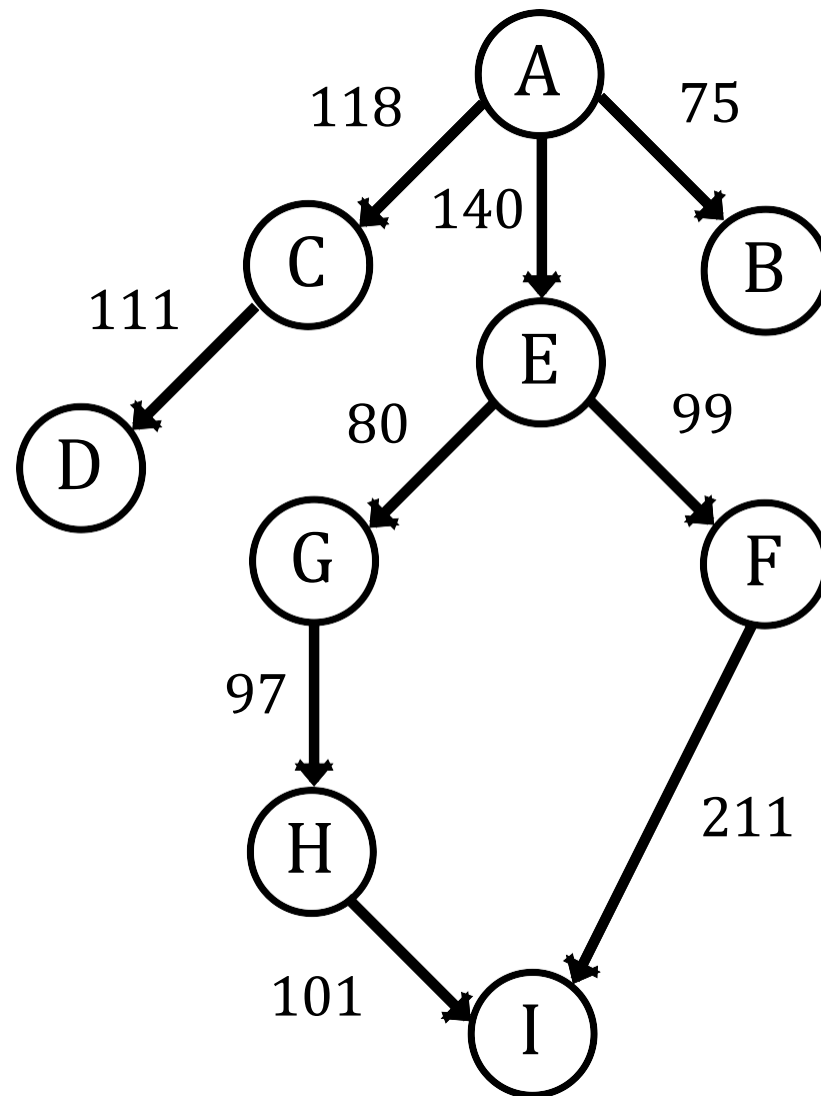
■ I = Goal State



A* Search (Problem 01)

$h(n)$ = Straight Line Distance

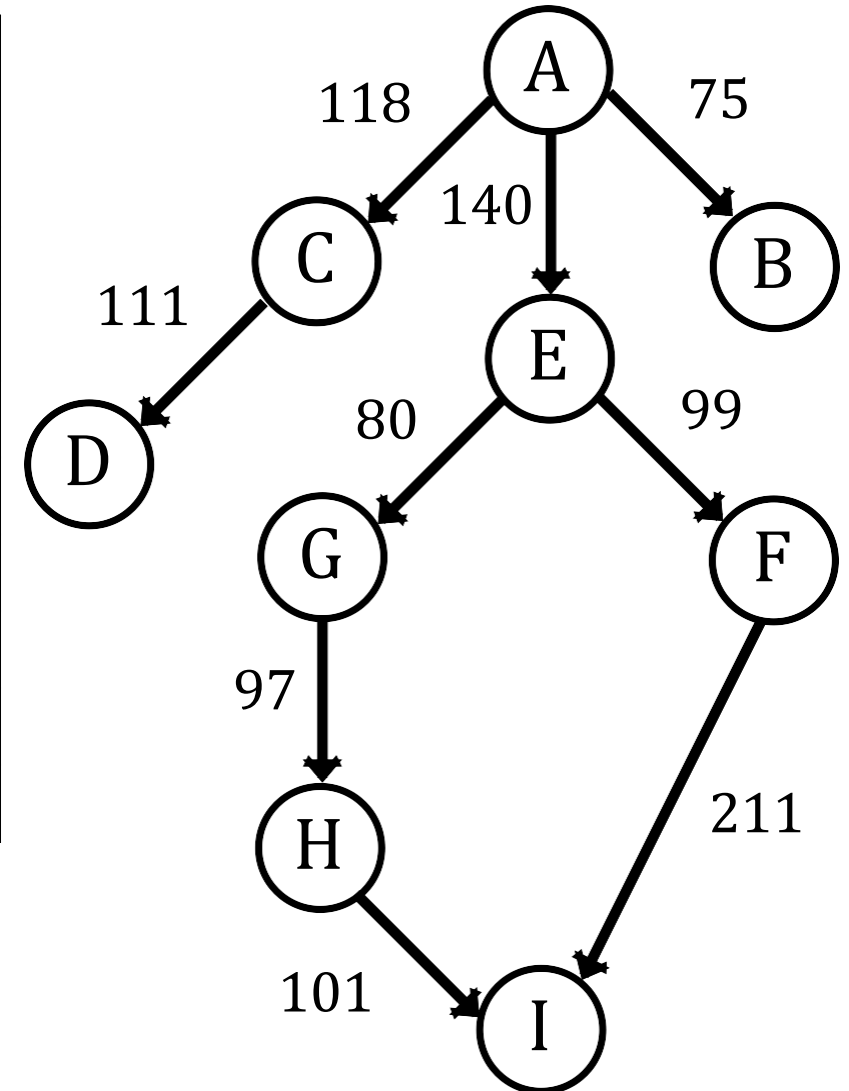
State	Heuristic: $h(n)$
A	366
B	374
C	329
D	244
E	253
F	178
G	193
H	98
I	0



A* Search (Problem 01)

Queue			
Path	g(n)	h(n)	f(n)

State	Heuristic: h(n)
A	366
B	374
C	329
D	244
E	253
F	178
G	193
H	98
I	0

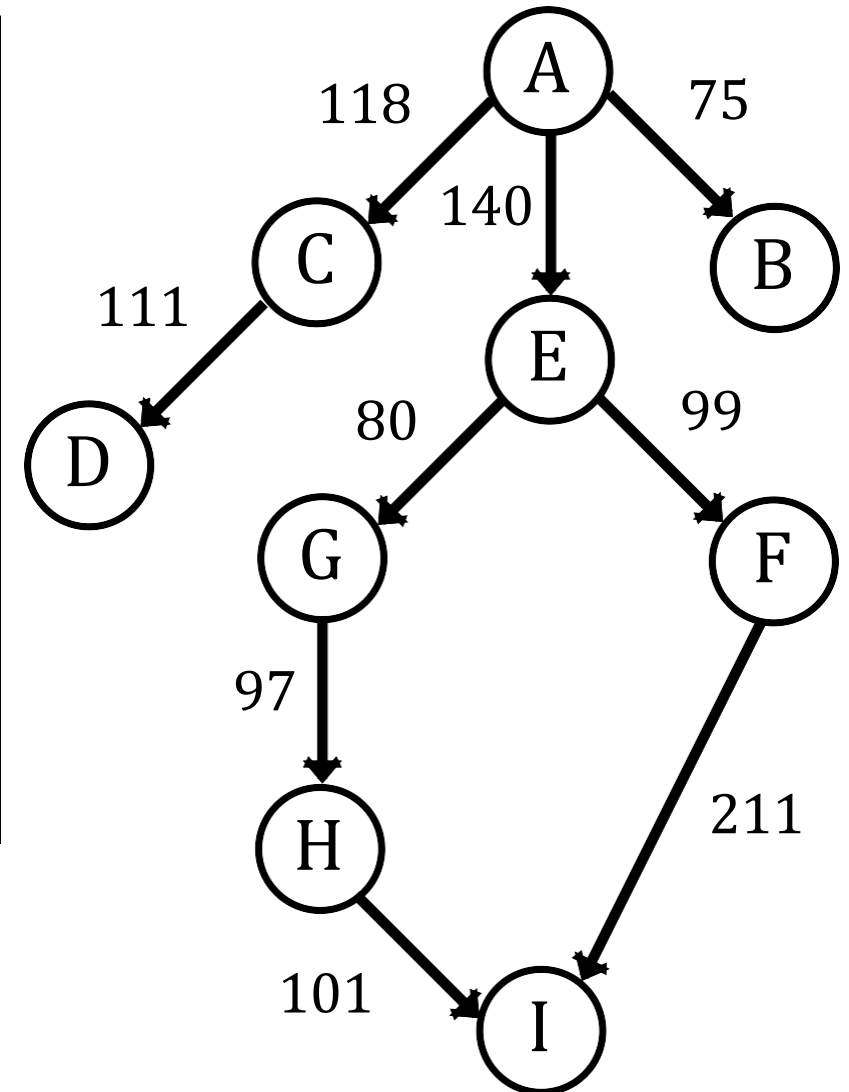


$$f(n) = g(n) + h(n)$$

A* Search (Problem 01)

Queue			
Path	g(n)	h(n)	f(n)
<A>	0	366	366

State	Heuristic: h(n)
A	366
B	374
C	329
D	244
E	253
F	178
G	193
H	98
I	0

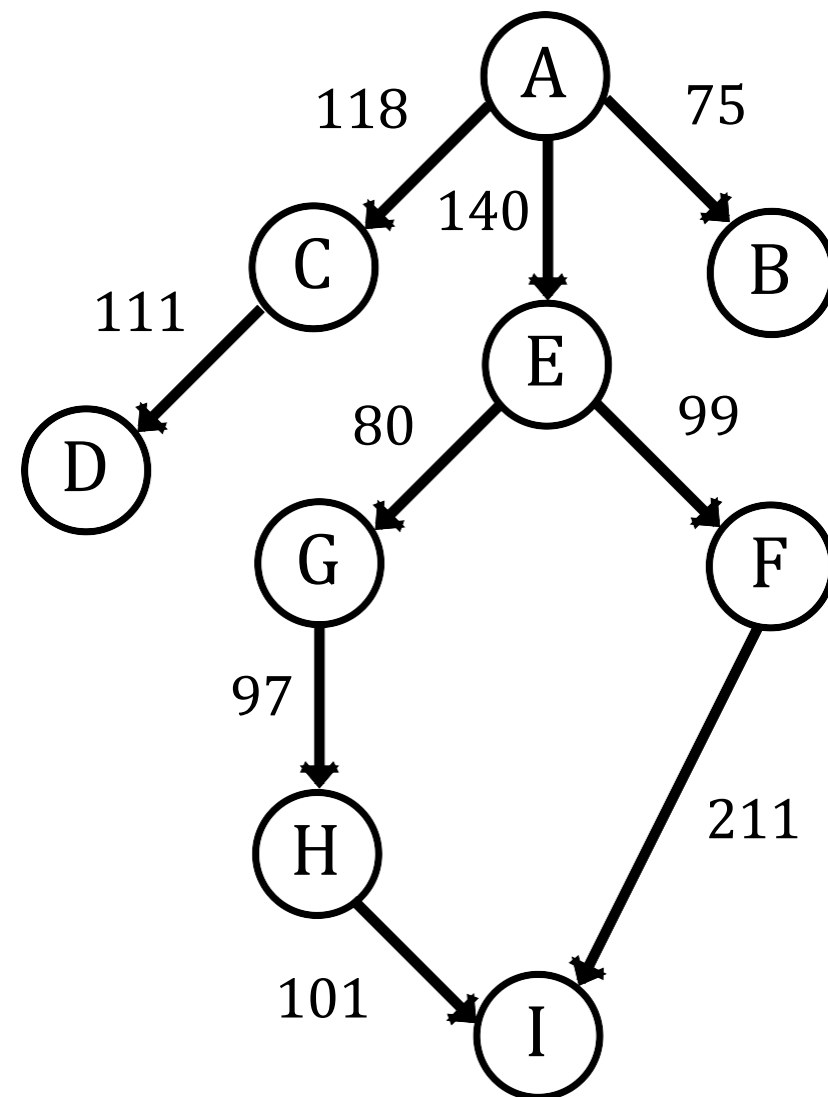


$$f(n) = g(n) + h(n)$$

A* Search (Problem 01)

Queue			
Path	g(n)	h(n)	f(n)
<E,A>	140	253	393
<C,A>	118	329	447
<B,A>	75	374	449

State	Heuristic: h(n)
A	366
B	374
C	329
D	244
E	253
F	178
G	193
H	98
I	0

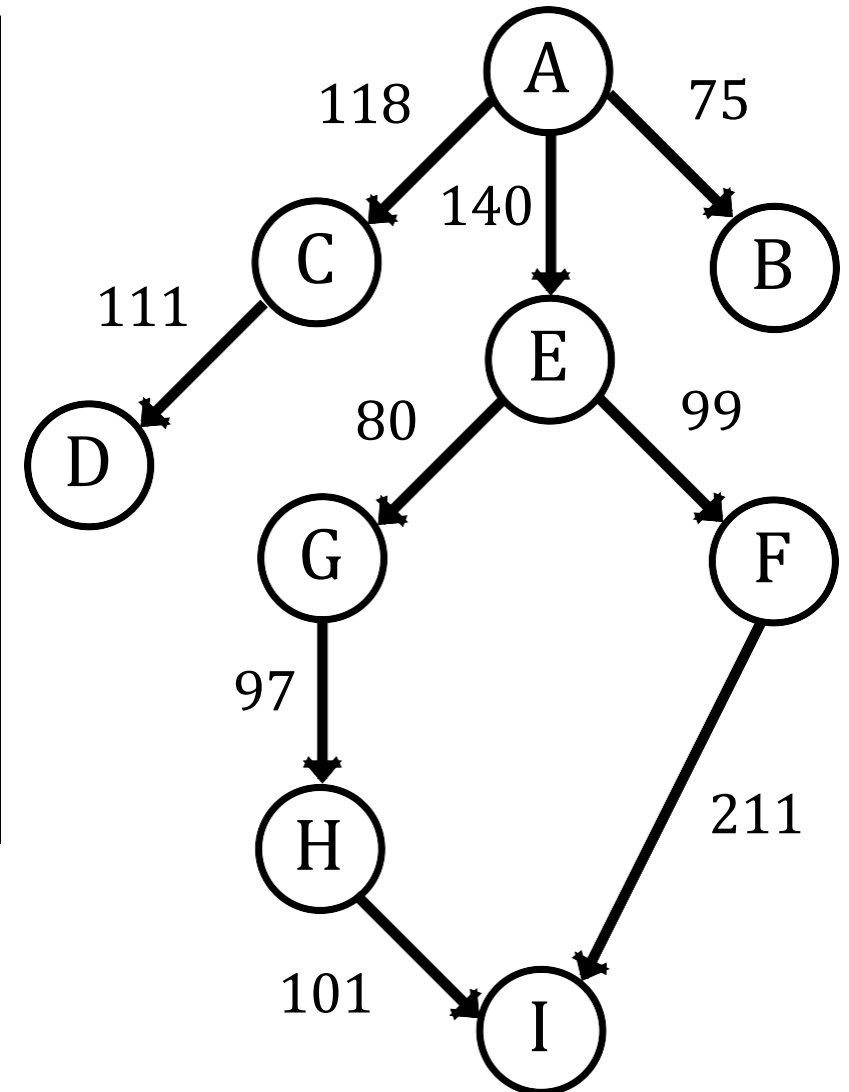


$$f(n) = g(n) + h(n)$$

A* Search (Problem 01)

Queue			
Path	g(n)	h(n)	f(n)
<G,E,A>	220	193	413
<F,E,A>	239	178	417
<C,A>	118	329	447
<B,A>	75	374	449

State	Heuristic: h(n)
A	366
B	374
C	329
D	244
E	253
F	178
G	193
H	98
I	0

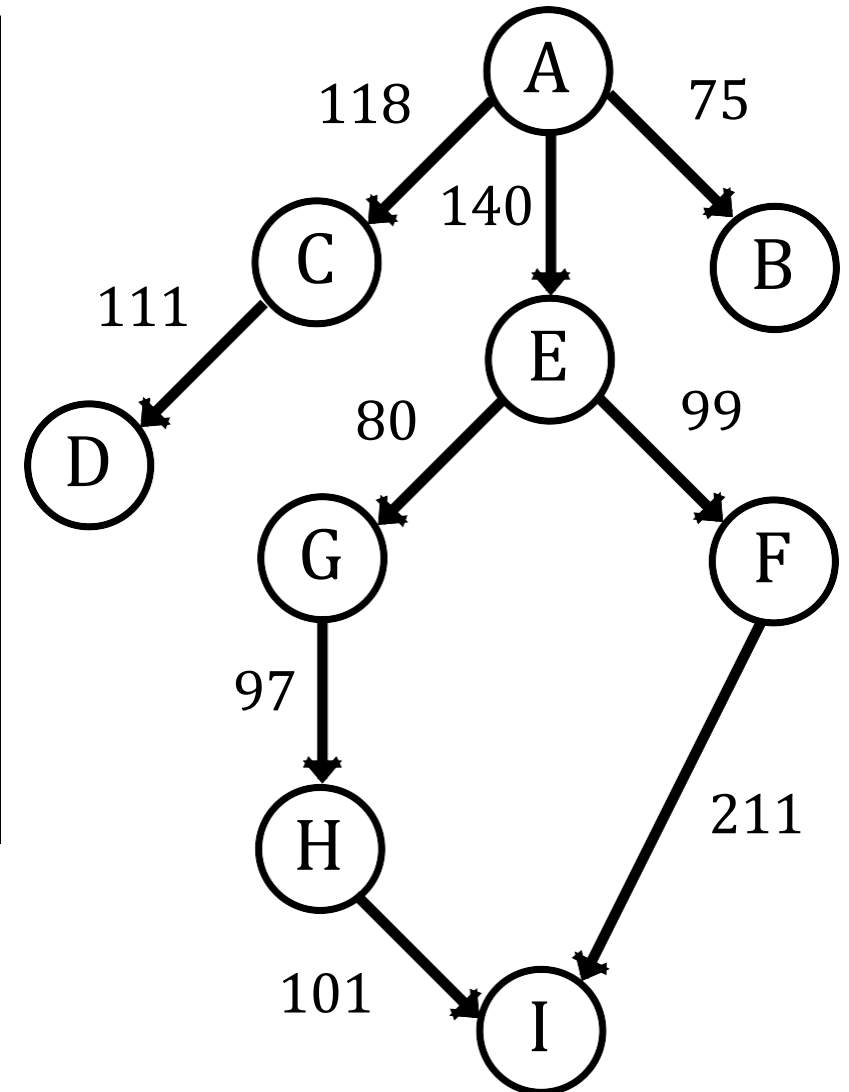


$$f(n) = g(n) + h(n)$$

A* Search (Problem 01)

Queue			
Path	g(n)	h(n)	f(n)
<H,G,E,A>	317	98	415
<F,E,A>	239	178	417
<C,A>	118	329	447
<B,A>	75	374	449

State	Heuristic: h(n)
A	366
B	374
C	329
D	244
E	253
F	178
G	193
H	98
I	0

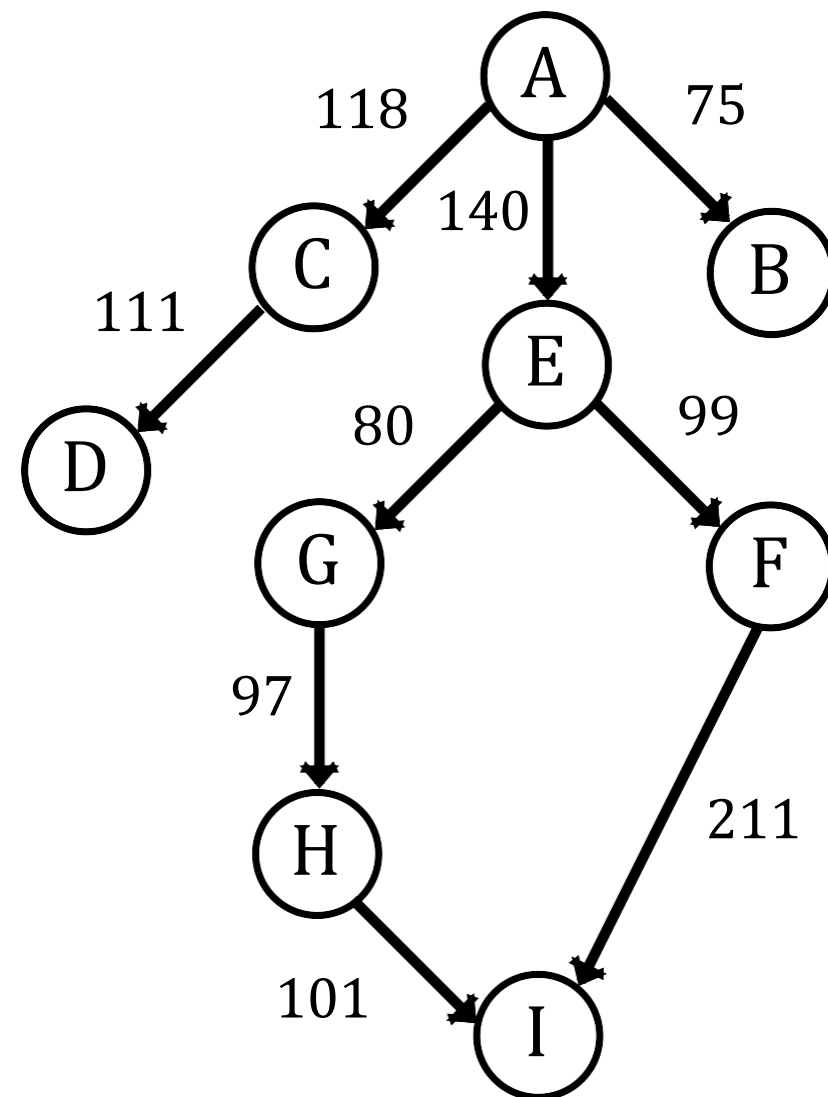


$$f(n) = g(n) + h(n)$$

A* Search (Problem 01)

Queue			
Path	g(n)	h(n)	f(n)
<F,E,A>	239	178	417
<I,H,G,E,A>	418	0	418
<C,A>	118	329	447
<B,A>	75	374	449

State	Heuristic: h(n)
A	366
B	374
C	329
D	244
E	253
F	178
G	193
H	98
I	0

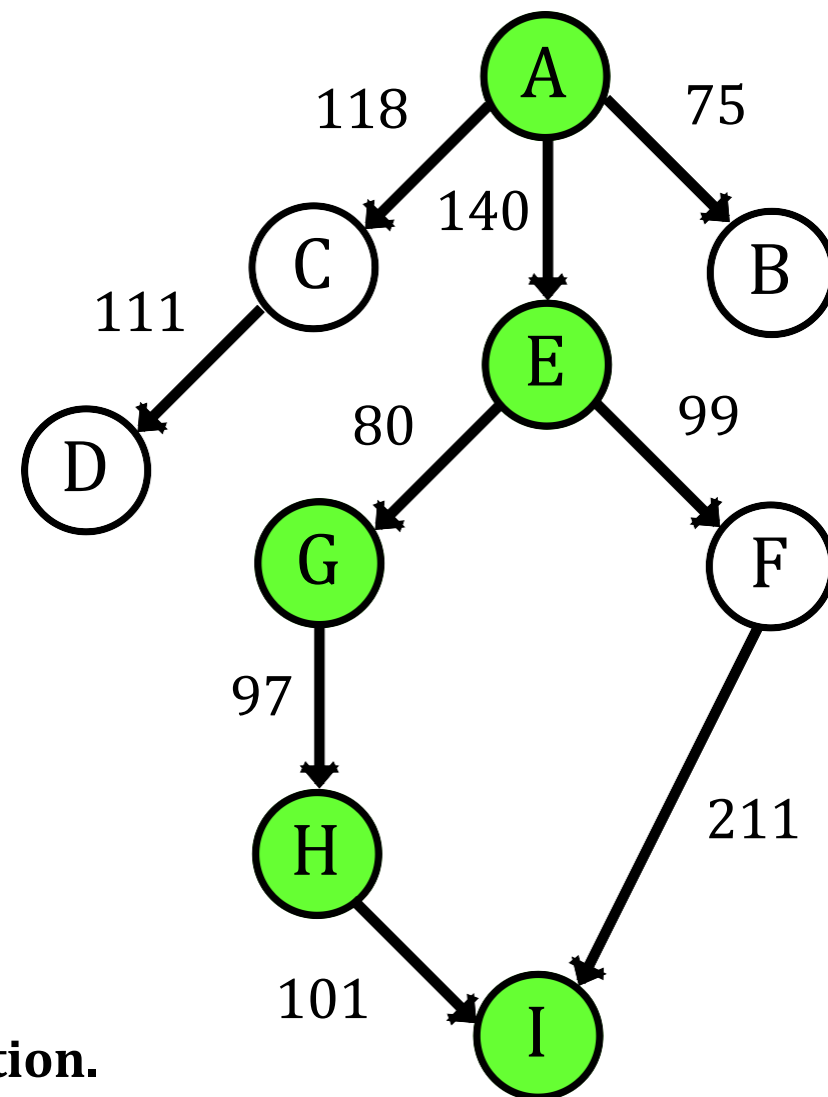


$$f(n) = g(n) + h(n)$$

A* Search (Problem 01)

Queue			
Path	g(n)	h(n)	f(n)
<I,H,G,E,A>	418	0	418
<C,A>	118	329	447
<B,A>	75	374	449
<I,F,E,A>	239	0	450

State	Heuristic: h(n)
A	366
B	374
C	329
D	244
E	253
F	178
G	193
H	98
I	0



Solution = A → E → G → H → I

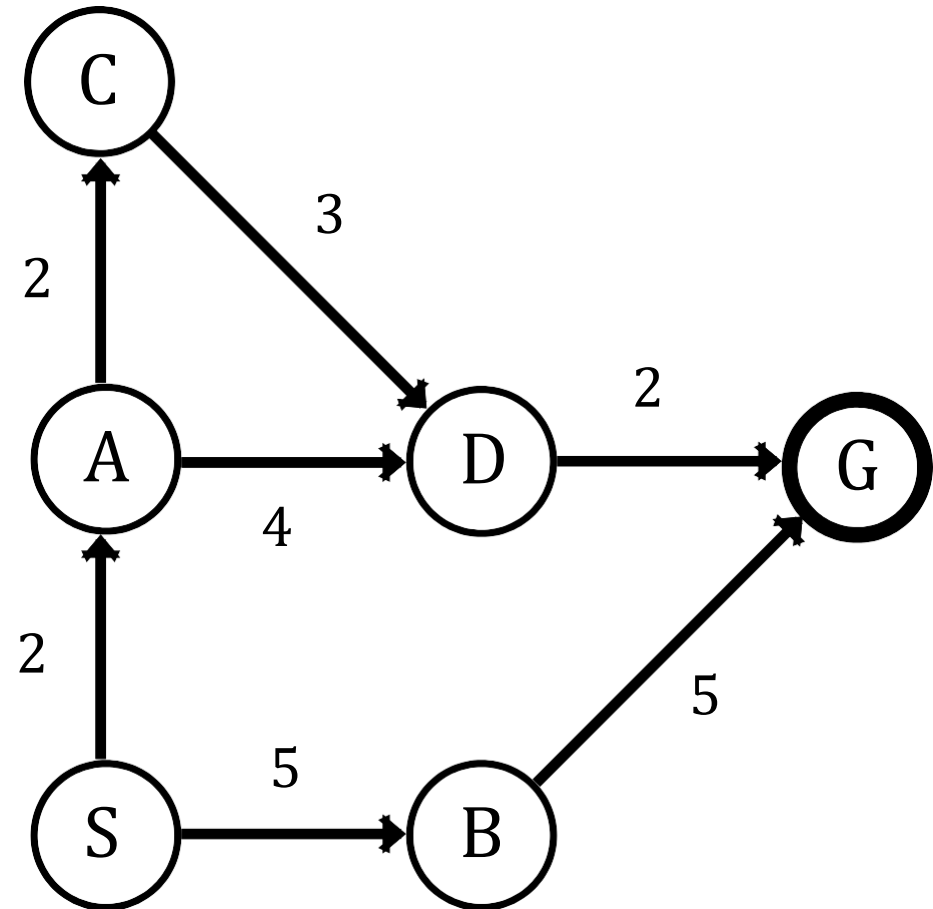
Path Cost = 140+80+97+101 = 418

As minimum path is the goal path, so we have found a solution.

A* Search (Problem 02)

■ **S = Initial State**

■ **G = Goal State**

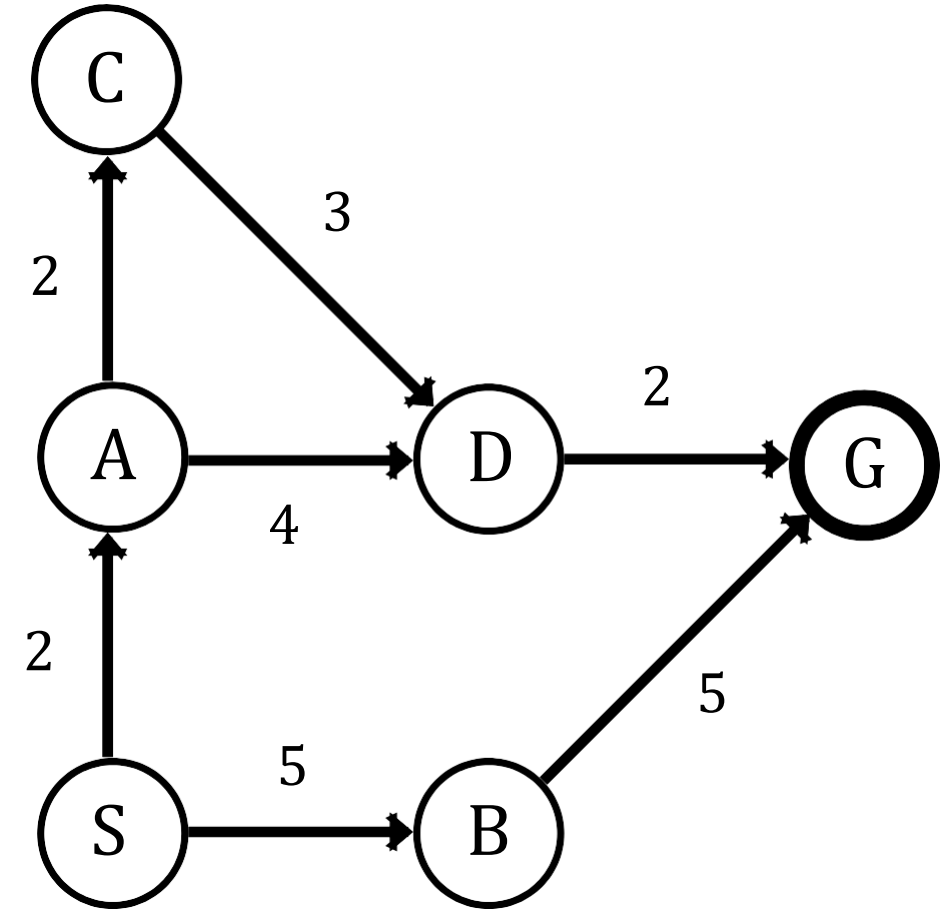


A* Search (Problem 02)

$f(n) = h(n) = \text{Straight Line Distance}$

State	Heuristic: $h(n)$
S	10
A	2
B	3
C	1
D	4
G	0

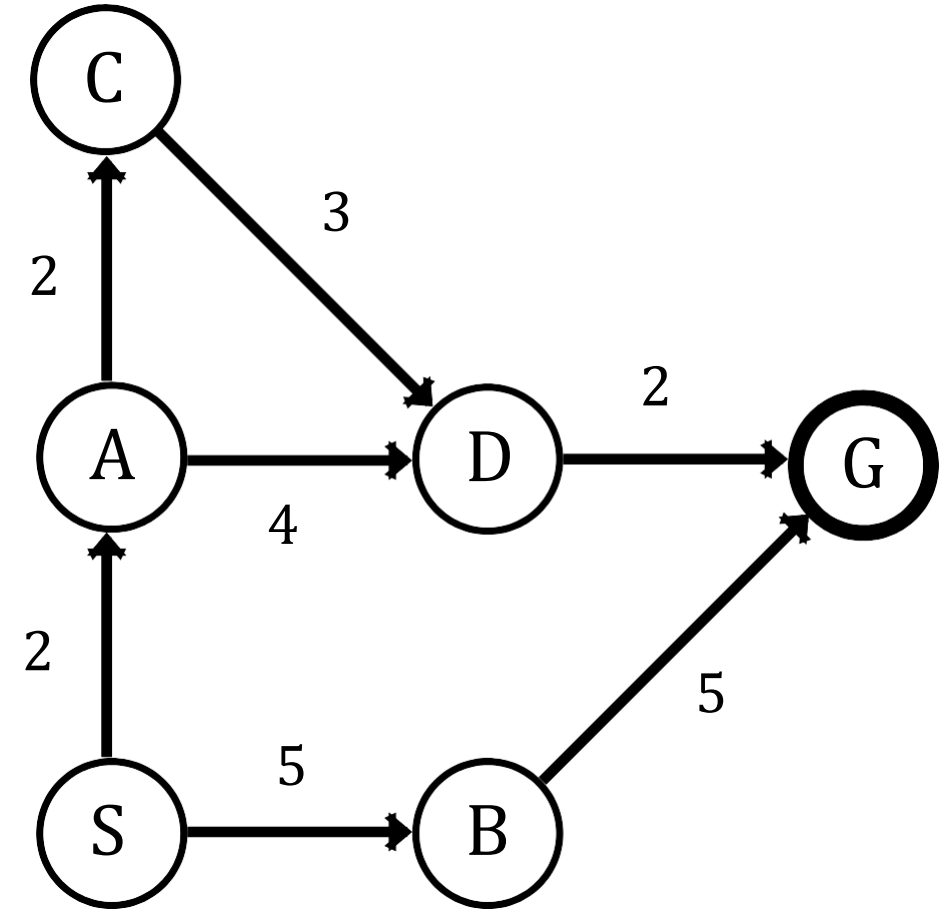
$$f(n) = g(n) + h(n)$$



A* Search (Problem 02)

Queue			
Path	g(n)	h(n)	f(n)

State	Heuristic: h(n)
S	10
A	2
B	3
C	1
D	4
G	0

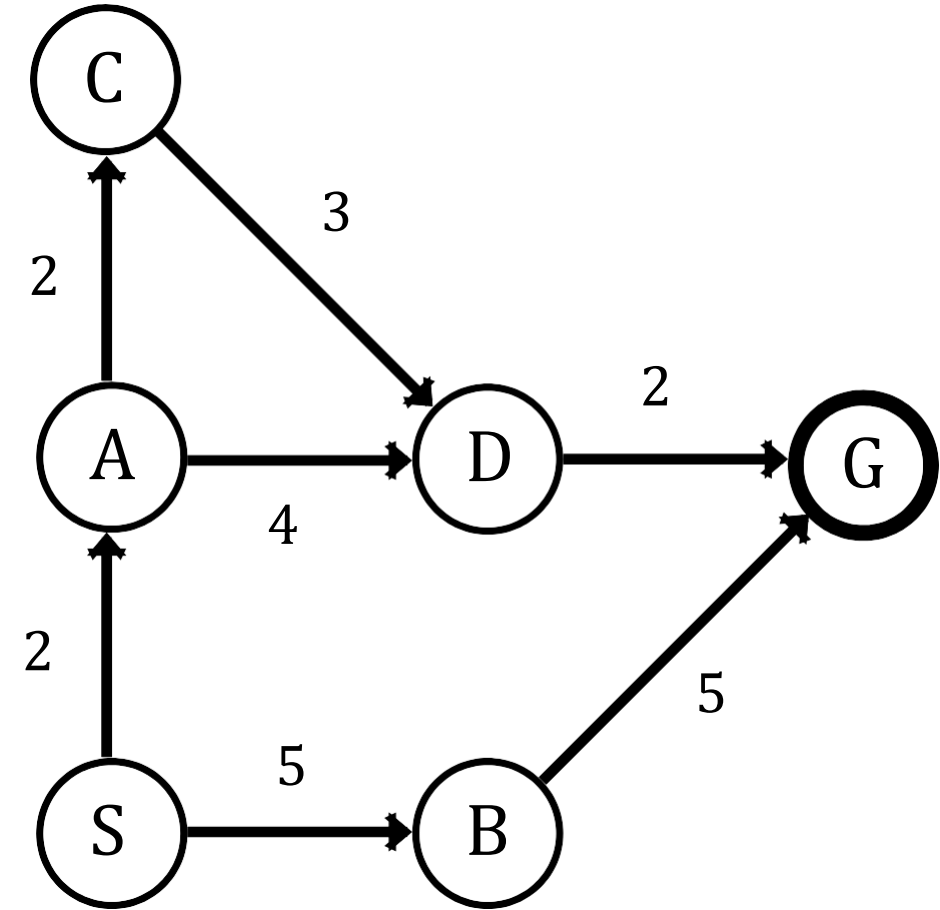


$$f(n) = g(n) + h(n)$$

A* Search (Problem 02)

Queue			
Path	$g(n)$	$h(n)$	$f(n)$
<S>	0	10	10

State	Heuristic: $h(n)$
S	10
A	2
B	3
C	1
D	4
G	0

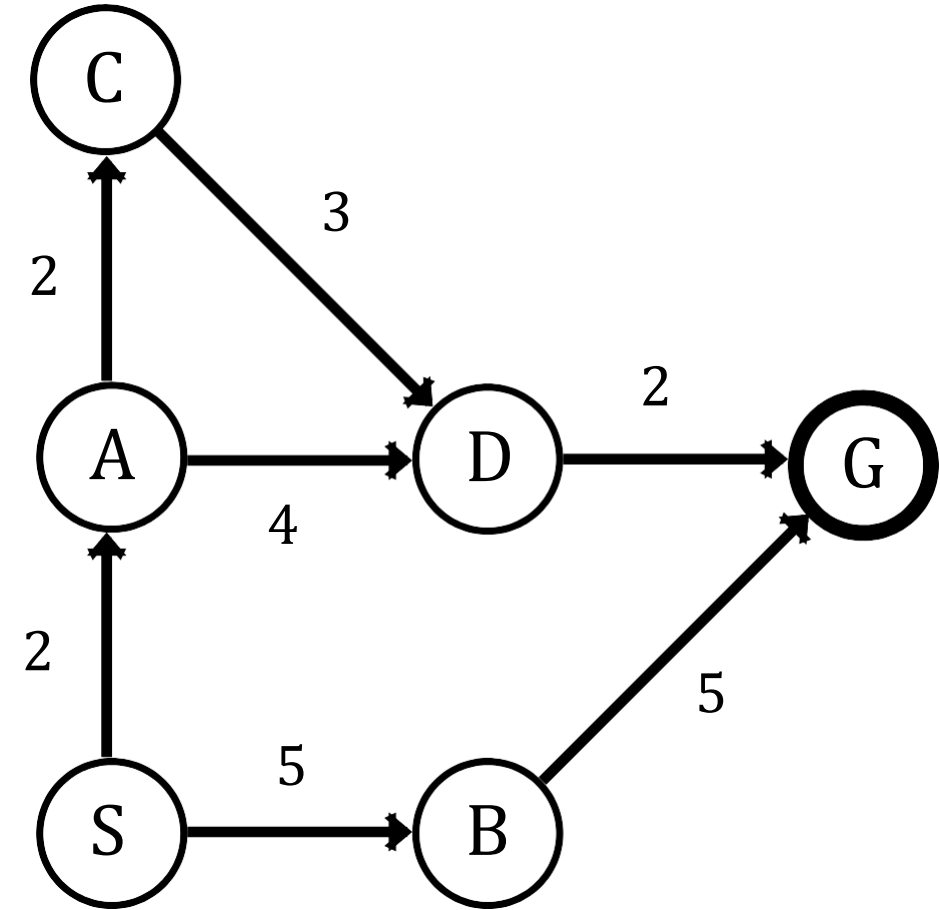


$$f(n) = g(n) + h(n)$$

A* Search (Problem 02)

Queue			
Path	$g(n)$	$h(n)$	$f(n)$
$\langle A, S \rangle$	2	2	4
$\langle B, S \rangle$	5	3	8

State	Heuristic: $h(n)$
S	10
A	2
B	3
C	1
D	4
G	0

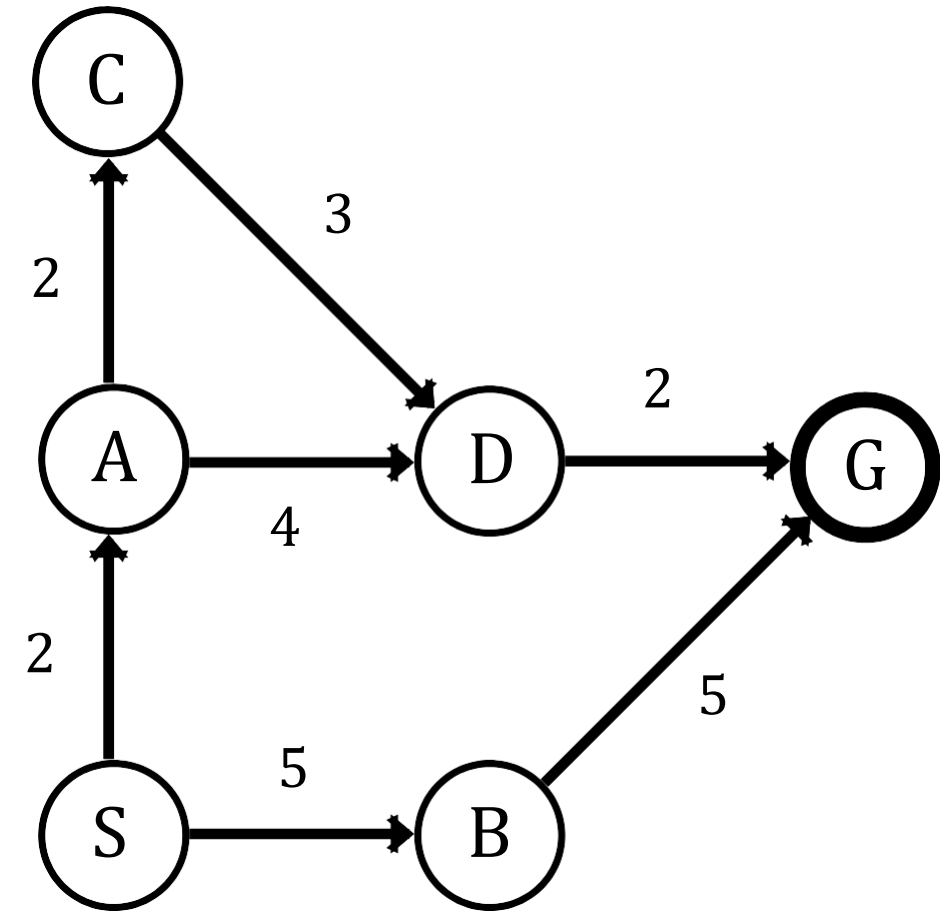


$$f(n) = g(n) + h(n)$$

A* Search (Problem 02)

Queue			
Path	g(n)	h(n)	f(n)
<C,A,S>	4	1	5
<B,S>	5	3	8
<D,A,S>	6	4	10

State	Heuristic: h(n)
S	10
A	2
B	3
C	1
D	4
G	0

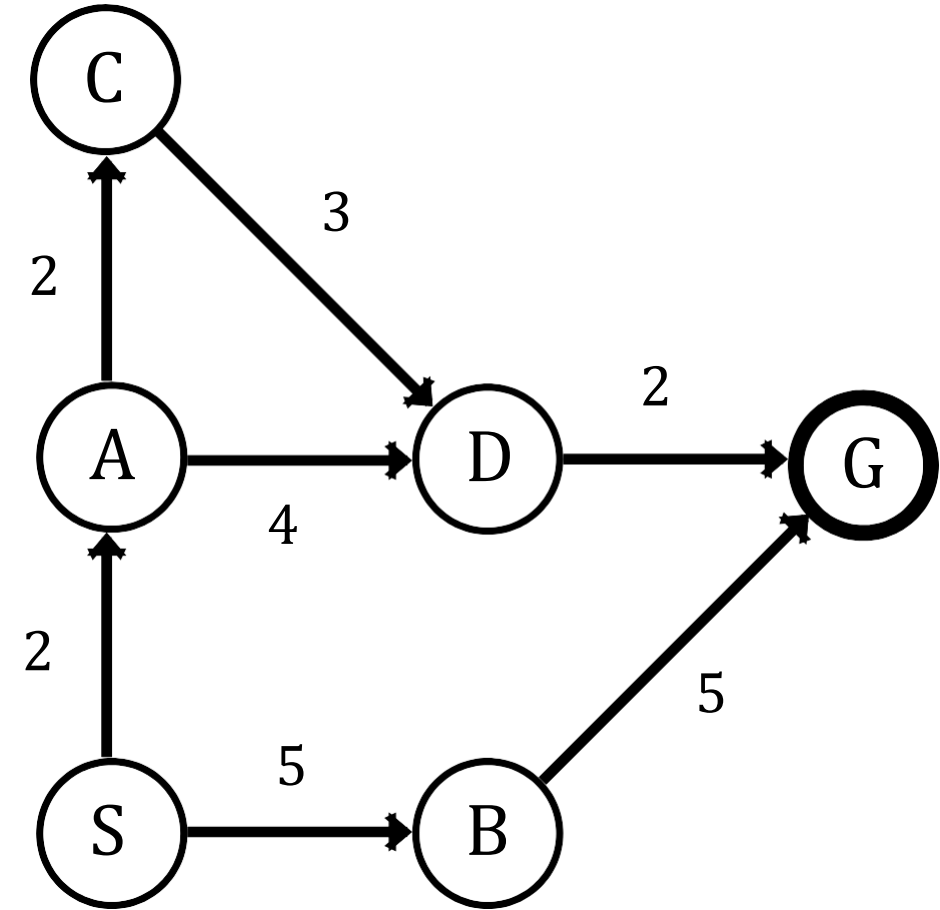


$$f(n) = g(n) + h(n)$$

A* Search (Problem 02)

Queue			
Path	$g(n)$	$h(n)$	$f(n)$
$\langle B, S \rangle$	5	3	8
$\langle D, A, S \rangle$	6	4	10
$\langle D, C, A, S \rangle$	7	4	11

State	Heuristic: $h(n)$
S	10
A	2
B	3
C	1
D	4
G	0

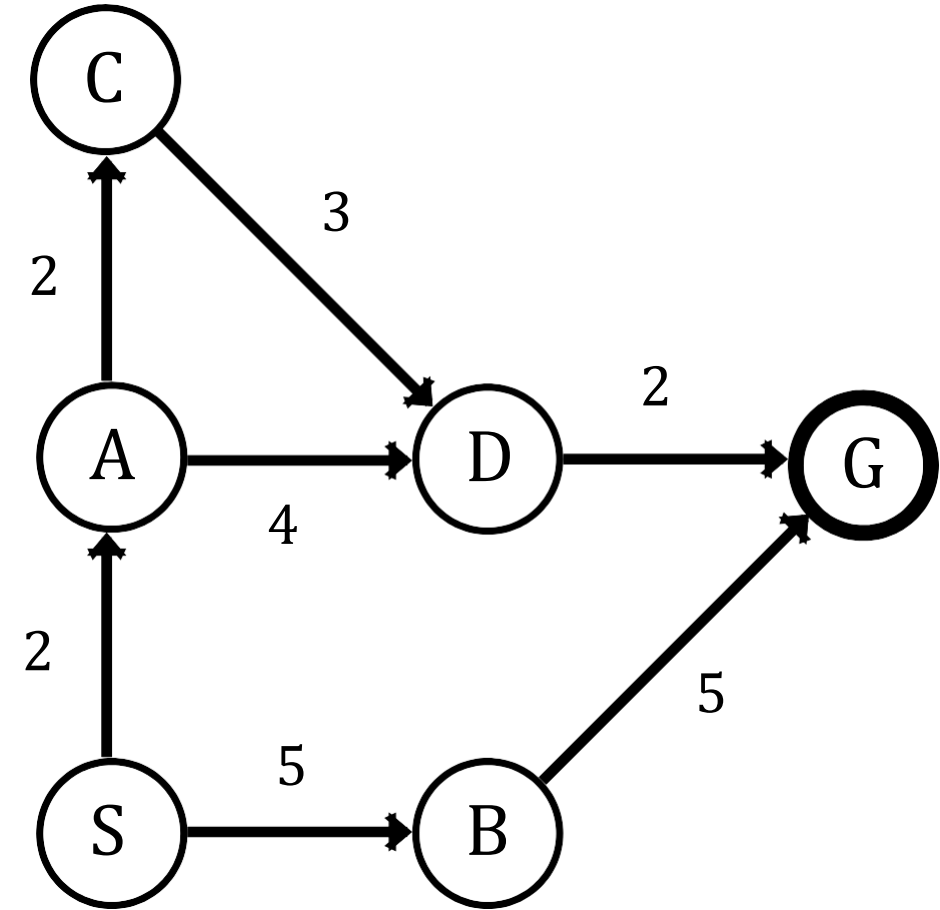


$$f(n) = g(n) + h(n)$$

A* Search (Problem 02)

Queue			
Path	g(n)	h(n)	f(n)
<D,A,S>	6	4	10
<G,B,S>	10	0	10
<D,C,A,S>	7	4	11

State	Heuristic: h(n)
S	10
A	2
B	3
C	1
D	4
G	0

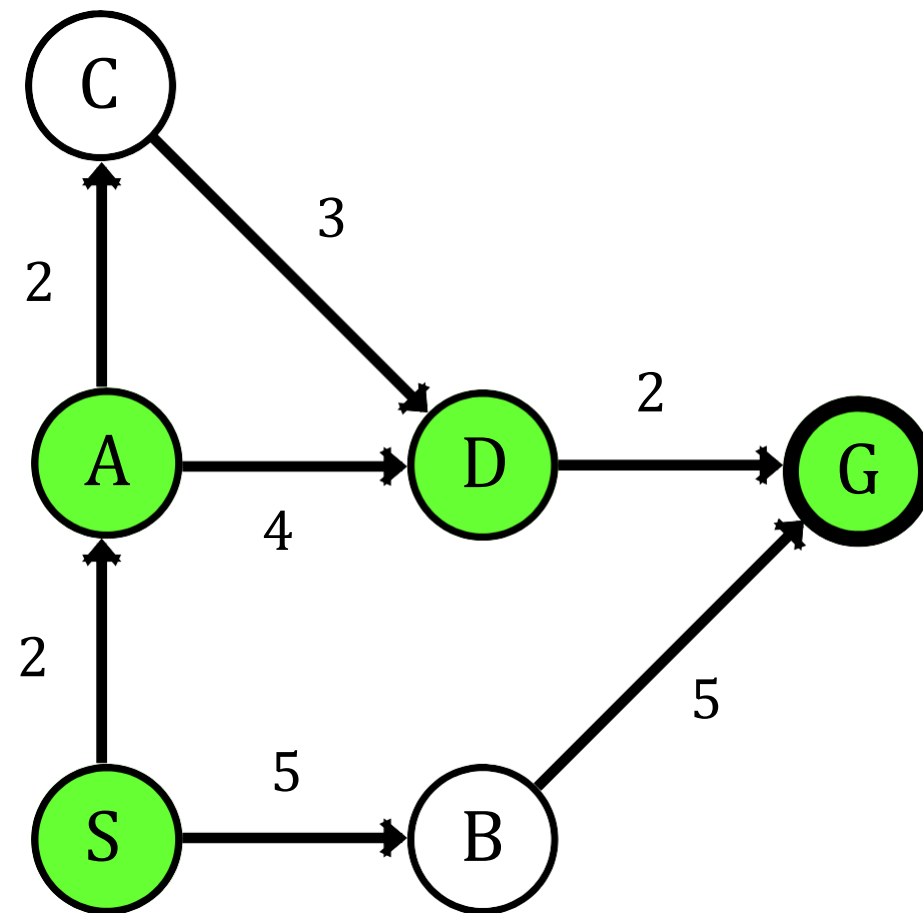


$$f(n) = g(n) + h(n)$$

A* Search (Problem 02)

Queue			
Path	g(n)	h(n)	f(n)
<G,D,A,S>	8	0	8
<G,B,S>	10	0	10
<D,C,A,S>	7	4	11

State	Heuristic: h(n)
S	10
A	2
B	3
C	1
D	4
G	0



Solution = $S \rightarrow A \rightarrow D \rightarrow G$

Path Cost = $2 + 4 + 2 = 8$

As minimum path is the goal path, so we have found a solution.

Admissible/Optimal Heuristics

- A heuristic h is *admissible* (optimistic) if:

$$0 \leq h(n) \leq h^*(n)$$

where $h^*(n)$ is the true cost to the nearest goal

Iterative Deepening A*

- Iterative-Deepening A* is a combination of Iterative Deepening Depth-First Search (IDDFS) and A* Search algorithms.
- IDA* performs a depth-limited search, gradually increasing the depth limit until a solution is found. At each iteration, it uses A* Search with the current depth limit.
- It maintains a threshold value, which is the minimum cost of the heuristic plus the cost of reaching the current state. If the estimated cost exceeds the threshold, the search is pruned.
- IDA* is memory-efficient as it only stores information for the current path, making it suitable for problems with large state spaces.
- It is complete and optimal if the heuristic is admissible.

Bidirectional A*

- Bidirectional A* simultaneously explores the search space from both the start state and the goal state, aiming to meet in the middle.
- It uses two heuristic evaluations, one from the start state to estimate the cost to reach each state, and another from the goal state to estimate the remaining cost to reach the goal.
- Bidirectional A* continues expanding nodes from both directions until they meet in the middle, or until the cost of reaching any intermediate node exceeds the current best solution.
- It can be more efficient than traditional A* Search, especially in scenarios where the search space is large and there are multiple paths to the goal.

Recursive Best-First Search

- Recursive Best-First Search is a memory-efficient variant of Best-First Search that avoids storing the entire search tree in memory.
- It recursively explores the search space, maintaining only a limited amount of information in memory at any given time.
- At each recursion level, RBFS keeps track of the f -value (estimated total cost) of the best alternative path found so far. If a better path is discovered, it updates the f -value and backtracks to explore other nodes.
- RBFS continues this process until a solution is found or until the entire search space is explored.
- It is complete and optimal under certain conditions, but its performance heavily depends on the quality of the heuristic function and the structure of the search space.

Algorithm	Completeness	Optimality	Time Complexity	Space Complexity	Frontier
Greedy Best First Search	Not guaranteed	Not guaranteed	$O(b^m)$ in worst case	$O(b^m)$ in worst case	Priority Queue based on heuristic value
A* Search	Guaranteed (if admissible heuristic)	Guaranteed (if admissible heuristic)	$O(b^d)$ in worst case	$O(b^d)$ in worst case	Priority Queue based on f-value (cost + heuristic value)
Iterative-Deepening A*	Complete	Optimal (under certain conditions)	$O(b^{\max(d,m)})$ in worst case (depends on heuristic)	Linear in space complexity	Priority Queue based on f-value (cost + heuristic value)

Algorithm	Completeness	Optimality	Time Complexity	Space Complexity	Frontier
Bidirectional A*	Complete (if both directions meet)	Optimal (under certain conditions)	$O(b^{d/2})$ in worst case (depends on heuristic)	Exponential in worst case (depends on heuristic)	Two Priority Queues (start and goal directions) based on f-value
Recursive Best-First Search	Complete	Optimal (under certain conditions)	$O(b^m)$ in worst case (depends on heuristic)	Linear in space complexity	Priority Queue based on f-value (cost + heuristic value)

- ***b***: Branching factor.
- ***d***: Depth of the optimal solution.
- ***m***: Maximum depth of the search space.

Informed Search Evaluation

- Greedy Best First Search is not guaranteed to be complete, as it may get stuck in cycles or choose paths that do not lead to the goal.
- Greedy Best First Search is not generally optimal, as it selects paths solely based on the heuristic value without considering the total cost.
- A* Search is complete if the heuristic function is admissible (never overestimates the true cost).
- A* Search is optimal if the heuristic function is admissible, ensuring that it finds the lowest-cost solution.

Real Life Examples of Informed Search

1. Greedy Best First Search:

- **Routing in Maps:** Finding the quickest route between two locations using a navigation app. Greedy Best First Search can prioritize nodes based on their proximity to the destination without considering traffic conditions or alternate routes.
- **Web Crawling:** Indexing web pages for a search engine. Greedy Best First Search can prioritize pages based on their relevance or popularity without considering the entire web graph.

2. A* Search:

- **Robot Path Planning:** Planning the path of a robot through an environment with obstacles to reach a goal. A* Search can efficiently find the shortest path by considering both the distance traveled and the estimated remaining distance to the goal.
- **Puzzle Solving:** Finding the optimal solution to puzzles such as the 15-puzzle or Rubik's Cube. A* Search can explore possible moves while guiding the search towards a solution using heuristic information.

3. Iterative Deepening:

- **Game Tree Search:** Searching for the optimal move in games like chess or checkers. Iterative Deepening can efficiently explore the game tree up to a certain depth, gradually increasing the depth limit until a solution is found.
- **Resource Allocation:** Optimizing resource allocation in project management or scheduling. Iterative Deepening can iteratively refine solutions by considering different allocations or schedules within increasing time bounds.

4. Bidirectional A*:

- **Social Network Analysis:** Finding the shortest path between two individuals in a social network. Bidirectional A* can simultaneously explore connections from both individuals, potentially reducing search time compared to unidirectional search.
- **Graph Search:** Finding the optimal route between two cities in a transportation network. Bidirectional A* can explore routes from both the origin and destination simultaneously, converging towards the optimal solution faster.

5. Recursive Best First Search:

- **Resource Allocation:** Optimizing resource allocation in complex projects where dependencies and constraints exist. Recursive Best First Search can efficiently explore alternative allocations while backtracking to explore more promising paths.
- **Route Planning with Dynamic Traffic:** Planning routes in urban areas with dynamically changing traffic conditions. Recursive Best First Search can adaptively explore routes based on real-time traffic updates while considering multiple alternatives.

Questions

1. Define "Informed Search" and contrast it with "Uninformed Search." How does the availability of additional information impact the search process?
2. Explore the significance of informed search algorithms in problem-solving. How do they balance the trade-off between exploration and exploitation of search space?
3. What is a "Heuristic" in the context of search algorithms? How does a heuristic provide additional knowledge to guide the search towards the goal?
4. Explain the concept of "Heuristic Search." How does a heuristic function help in guiding the search algorithm towards the goal state?
5. Describe the role of a "Heuristic Function" in informed search algorithms. How is a heuristic function typically defined, and what properties should it possess?
6. Define "Heuristic Evaluation." How do we measure the quality or effectiveness of a heuristic function in guiding the search towards the goal state?
7. Discuss the limitations of heuristic-based search algorithms. In what scenarios might heuristic information lead to suboptimal solutions or inefficiencies in the search process?
8. Differentiate between "Best First Search" and "Greedy Best First Search." What are the main characteristics of each algorithm, and how do they use heuristic information?
9. Explain the "A* Search" algorithm. How does A* combine both the benefits of breadth-first and greedy search, and what makes it an effective informed search algorithm?
10. Compare and contrast "Admissible Heuristics" and "Inadmissible Heuristics." What criteria must an admissible heuristic satisfy, and what are the implications of using an inadmissible heuristic in informed search algorithms?
11. Explain the concept of "Consistency" or "Monotonicity" in heuristic functions. How does consistency ensure the optimality of solutions in algorithms like A* Search?
12. Investigate the role of "Domain Knowledge" in developing effective heuristic functions. How does domain-specific knowledge influence the design and performance of informed search algorithms?
13. Analyze the computational complexity of informed search algorithms such as A* Search. How do factors like the size of the search space and the quality of the heuristic affect the algorithm's time and space complexity?
14. Explore the concept of "Optimality Bounds" in heuristic-based search algorithms. How do we establish bounds on the optimality of solutions produced by algorithms like A* Search?

Questions Cont'd

15. What is the main idea behind Iterative Deepening A*?
16. How does IDA* combine the benefits of Iterative Deepening Depth-First Search (IDDFS) and A* Search?
17. Explain the role of the heuristic function in IDA*.
18. What are the advantages and disadvantages of IDA* compared to A* Search?
19. Can IDA* guarantee optimality under certain conditions? If yes, what are those conditions?
20. Describe the concept of Bidirectional A*.
21. How does Bi-A* differ from traditional A* Search in terms of exploration strategy?
22. What are the potential advantages of using Bi-A* over A* Search?
23. What challenges might arise when implementing Bi-A* in practice?
24. Under what conditions can Bi-A* guarantee completeness and optimality?
25. What is the main motivation behind Recursive Best-First Search?
26. How does RBFS manage memory more efficiently compared to traditional Best-First Search?
27. Explain the concept of backtracking in RBFS.
28. What role does the evaluation function play in RBFS?
29. Can RBFS guarantee optimality under certain conditions? If yes, what are those conditions?
30. Provide real-life examples where informed search algorithms are used. How do these examples demonstrate the practical applications of heuristic-based search techniques in various domains?

Programming Tasks

1. Implement heuristic functions for the Eight Puzzle problem. Write functions that estimate the cost from a given state to the goal state based on heuristics such as Manhattan distance or Hamming distance. Example: Implement a function `manhattan_distance(state, goal_state)` that calculates the Manhattan distance heuristic for the Eight Puzzle.
2. Implement the Best First Search algorithm using a priority queue. Use a heuristic function to guide the search towards the goal state. Example: Write a function `best_first_search(initial_state, goal_state, heuristic_func)` that performs Best First Search on a given problem domain using the specified heuristic function.
3. Implement the Greedy Best First Search algorithm, which prioritizes expanding nodes based solely on heuristic values. Example: Implement a function `greedy_best_first_search(initial_state, goal_state, heuristic_func)` that performs Greedy Best First Search using the specified heuristic function.
4. Implement the A* Search algorithm using a priority queue. Utilize both the heuristic function and the cost function to guide the search towards the goal state while ensuring optimality. Example: Write a function `a_star_search(initial_state, goal_state, heuristic_func, cost_func)` that performs A* Search on a given problem domain using the specified heuristic and cost functions.
5. Choose a problem domain (e.g., pathfinding, puzzle-solving) and implement a solution using an informed search algorithm such as A* Search. Use appropriate heuristic functions tailored to the problem domain. Example: Implement a program that solves the Traveling Salesman Problem using A* Search with a custom heuristic function based on minimum spanning trees.
6. Implement multiple search algorithms (e.g., BFS, DFS, A* Search) for the same problem domain and compare their performance in terms of solution quality, computational time, and space complexity. Example: Write functions for BFS, DFS, and A* Search and evaluate their performance on various graphs or problem instances, recording metrics such as solution length, execution time, and memory usage.
7. Experiment with different heuristic functions for the same problem domain and evaluate their effectiveness in guiding the search process. Optimize the heuristic function to improve the efficiency of the informed search algorithm. Example: Implement a program that solves the Eight Puzzle using A* Search and test different heuristic functions (e.g., Manhattan distance, Hamming distance) to find the most effective one.
8. Develop a simple navigation system using an informed search algorithm (e.g., A* Search) to find the shortest path between two locations on a map. Utilize real geographic data and heuristic functions based on distance or travel time. Example: Create a Python script that takes a map graph as input, along with start and end locations, and uses A* Search with a heuristic based on Euclidean distance to find the shortest path.
9. Create a program that solves a puzzle (e.g., Eight Puzzle, Sudoku) using an informed search algorithm. Implement heuristic functions tailored to the specific puzzle constraints to guide the search towards a solution. Example: Develop a Sudoku solver using A* Search with a heuristic function based on the number of filled cells in each row, column, and box.

Programming Tasks Cont'd

10. Implement the IDA* algorithm to solve the 8-puzzle problem. Use a suitable heuristic function (e.g., Manhattan distance) to guide the search.
11. Create a visualization of the IDA* algorithm's progress, showing the states explored and the path taken to reach the goal state.
12. Develop a Bi-A* algorithm to find the shortest path between two nodes in a weighted graph. Test it on a variety of graphs, including directed and undirected graphs with different edge weights.
13. Extend the Bi-A* algorithm to handle graphs with dynamic edge weights. Implement appropriate update mechanisms to adapt to changes in the graph during the search process.
14. Write a Python function to perform RBFS on a tree structure represented as a dictionary or an object-oriented data structure. Use a suitable evaluation function to guide the search.
15. Implement RBFS to solve the N-Queens problem, where the goal is to place N queens on an $N \times N$ chessboard without any queen attacking each other. Use RBFS to explore the solution space efficiently.